

PROJECT_FULL_DOCUMENTATION.md

Limitless Future — AI Forklift Safety System

Complete Technical Reference for Maintenance, Development & Operations Teams

Document Version: 1.0 | Generated: 2026-07-16 | Target Platform: Ubuntu 24.04 LTS

SCOPE: This document is the authoritative technical reference for the entire AI_Forklift_Safety project. It covers architecture, every source file, every class, every function, threading, hardware protocols, security, performance, and known issues. A new engineer with no prior knowledge of this project should be able to fully understand, maintain, debug, and extend the system using only this document.

TABLE OF CONTENTS

- 1. [Section 1 — Project Overview](#)
- 2. [Section 2 — Project Architecture](#)
- 3. [Section 3 — Directory Tree](#)
- 4. [Section 4 — File-by-File Analysis](#)
- 5. [Section 5 — Class Analysis](#)
- 6. [Section 6 — Function Analysis](#)
- 7. [Section 7 — Line-by-Line Explanation](#)
- 8. [Section 8 — Call Flow](#)
- 9. [Section 9 — Dependency Graph](#)
- 10. [Section 10 — Configuration](#)
- 11. [Section 11 — AI Module](#)
- 12. [Section 12 — Hardware / Relay Module](#)
- 13. [Section 13 — User Interface \(UI\)](#)
- 14. [Section 14 — Threads](#)
- 15. [Section 15 — Error Handling](#)
- 16. [Section 16 — Performance Analysis](#)
- 17. [Section 17 — Security Analysis](#)
- 18. [Section 18 — Known Issues](#)
- 19. [Section 19 — Future Improvements](#)
- 20. [Section 20 — Overall Project Health](#)
- 21. [Section 21 — Complete Variable Reference](#)
- 22. [Section 22 — Object Lifetime Analysis](#)

- 23. [Section 23 — Complete Function Call Graph](#)
- 24. [Section 24 — Variable Flow Analysis](#)
- 25. [Section 25 — Complete Data Flow](#)
- 26. [Section 26 — Complete Event Flow](#)
- 27. [Section 27 — Thread Interaction Analysis](#)
- 28. [Section 28 — Dependency Matrix](#)
- 29. [Section 29 — Configuration Usage Map](#)
- 30. [Section 30 — Memory Ownership Analysis](#)
- 31. [Section 31 — Recent Production-Grade Upgrades & System Resilience](#)

SECTION 1 — PROJECT OVERVIEW

1.1 Project Purpose

The **Limitless Future AI Forklift Safety System** is a production-grade, real-time industrial safety application designed for high-risk warehouse and manufacturing environments. Its primary mission is to **detect human presence in dangerous zones around operating forklifts** and immediately trigger a physical electrical relay (alarm/horn/emergency-stop) to alert workers and operators before a collision or injury occurs.

The system is marketed and engineered to meet **Pharma 4.0** industrial safety standards.

1.2 Main Objective

Prevent forklift-related injuries and fatalities by using AI computer vision to detect persons entering defined hazard zones around forklift operating paths, and immediately activate physical hardware alarms.

The system continuously monitors up to **4 camera feeds simultaneously**, each assigned to a unique industrial station, and uses a **YOLOv8 neural network** for real-time person detection. When a person's body center crosses into a pre-configured Region of Interest (ROI), the system sends a command through a hardware relay board to activate a physical alarm device (horn, siren, strobe light, or machine stop).

1.3 Current Development Status

Status: PRODUCTION DEPLOYED — The system is fully operational on Ubuntu 24.04 LTS hardware. It is deployed in a real industrial facility. It has: - A working 2-process IPC architecture (GUI + hardware daemon) - A complete unit test suite (13 test cases + stress test) - Role-based access control - ROI persistence - Automatic camera reconnection - Automatic relay board reconnection

1.4 Features Implemented

Feature	Status	Notes
YOLOv8 real-time person detection	✔ Complete	Model: <code>yolo8n.pt</code> (nano), COCO class 0
4-camera simultaneous monitoring	✔ Complete	2×2 grid layout

Feature	Status	Notes
Region of Interest (ROI) per camera	✓ Complete	Draw, move, resize, persist to JSON
Physical relay hardware control	✓ Complete	RM04U 4-Ch USB relay via serial ASCII
IPC Socket architecture	✓ Complete	Unix Domain Socket <code>/dev/shm/forklift_relay.sock</code>
OR-gate hardware logic	✓ Complete	Multiple stations can share one relay
Configurable relay mapping	✓ Complete	Each station → any of 4 relay channels
5-level role-based access control	✓ Complete	Operator/Technician/Supervisor/Engineer/Administrator
Password-protected role changes	✓ Complete	Stored in <code>security.json</code>
Auto-logout (5-minute inactivity)	✓ Complete	Reverts to Operator role
Boot self-test sequence	✓ Complete	Sequential ON/OFF all 4 relays on startup
Camera auto-reconnect	✓ Complete	3-second retry loop
Hardware daemon auto-reconnect	✓ Complete	3-second retry loop on serial loss
Frame skipping optimization	✓ Complete	YOLO inference every 3rd frame
Kiosk / Fullscreen mode	✓ Complete	FramelessWindowHint + WindowStaysOnTopHint
Audit log terminal	✓ Complete	Color-coded, timestamped, 50-entry ring buffer
Unit test suite	✓ Complete	13 test cases + 10,000-cycle stress test
Linux udev device symlink	✓ Complete	<code>/dev/forklift_relay</code> permanent name
Alt+F4 lockout	✓ Complete	Requires Supervisor role to close
Software coil enable/disable	✓ Complete	Hard-cuts all relay output
Manual relay test buttons	✓ Complete	Per-channel toggle in System Config page
On-delay / off-delay timers	✓ Complete (client-side)	Debounce only; server is timer-free
Simulated camera mode	✓ Complete (subpackage only)	<code>forklift_ai_safety/</code> refactored package
Physical USB Port Binding	✓ Complete	Anti-swapping kernel topology path resolution (<code>get_physical_usb_path</code>)
Per-Station Relay Binding & Live Test	✓ Complete	Dynamic ⚡ Relay: dropdown + instant ⚡ TEST R1-R4 buttons per card
Systemd Auto-Restart & Self-Healing	✓ Complete	<code>forklift-ai.service</code> (<code>Restart=always</code> , <code>RestartSec=3</code> , linger enabled)

Feature	Status	Notes
15-Second Hardware Stabilization Delay	✓ Complete	Live startup countdown in <code>start_system.sh</code> with <code>--no-delay</code> bypass
Layout Stretch & Aspect Ratio Protection	✓ Complete	50/50 grid stretch, <code>QSizePolicy.Ignored</code> , invariant <code>sizeHint()</code> , 640x480 banner
Normalized ROI Coordinates (0.0 to 1.0)	✓ Complete	Precision float ratio <code>(norm_x, norm_y, norm_w, norm_h)</code> with dynamic <code>resizeEvent</code>
Universal Responsive UI Scaling	✓ Complete	Adaptive <code>_ui_scale</code> [0.45, 1.50], tested clean from 800x480 up to 4K UHD

1.5 Missing Features / Not Yet Implemented

Missing Feature	Priority	Notes
Event recording / video clip saving	Medium	No incident clip archiving
Remote monitoring / SCADA integration	Medium	No OPC-UA, MQTT, or Modbus TCP
Alarm acknowledgement workflow	Medium	No ACK button; alarm auto-clears when zone is clear
Email/SMS notification	Low	No alerting to external systems
Multi-user simultaneous sessions	Not applicable	Kiosk mode is single-user by design
Database logging	Medium	Events only go to GUI terminal, not a DB
RTSP / IP Camera support	Partial	Code accepts string device paths but no tested RTSP
Camera stream recording	Low	No DVR capability

1.6 Technologies Used

Category	Technology	Version / Detail
OS	Ubuntu Linux	24.04 LTS (primary target)
Language	Python	3.10+
GUI Framework	PyQt5	Qt5 bindings for Python
AI/ML Framework	Ultralytics YOLO	YOLOv8
Computer Vision	OpenCV (<code>cv2</code>)	Frame capture, annotation, color conversion
Deep Learning	PyTorch (via ultralytics)	Backend for YOLO inference
Serial Communication	pyserial	USB-Serial to RM04U relay board
IPC	Unix Domain Sockets	<code>/dev/shm/forklift_relay.sock</code> (tmpfs)
Serialization	JSON	Config files, IPC protocol, security store
Testing	Python <code>unittest</code> + <code>pytest</code>	OR-gate logic tests
Hardware	RM04U 4-Channel USB Relay	CH340 chip, ASCII protocol

Category	Technology	Version / Detail
Hardware	V4L2 USB Cameras	Linux video4linux2 subsystem

1.7 AI Models

Model	File	Size	Architecture	Dataset	Classes Used
YOLOv8 Nano	<code>yolo8n.pt</code>	~6.5 MB	YOLOv8 CSP	COCO 80-class	Class 0 (person) only

1.8 Hardware Requirements

Component	Specification
CPU	x86-64, minimum 4 cores recommended (YOLO inference is CPU-bound on this build)
RAM	Minimum 4 GB; 8 GB recommended with 4 cameras active
GPU	Optional; PyTorch will use CPU if no CUDA GPU present
USB Relay Board	RM04U 4-Channel USB Relay (CH340 USB chip, VID:1a86 PID:7523)
Cameras	Up to 4 USB V4L2 cameras (even-numbered <code>/dev/video*</code> nodes: 0, 2, 4, 6)
Storage	10 GB minimum for OS + venv + YOLO weights
Display	Any HDMI/DP monitor; fullscreen kiosk assumes 1280×800 minimum

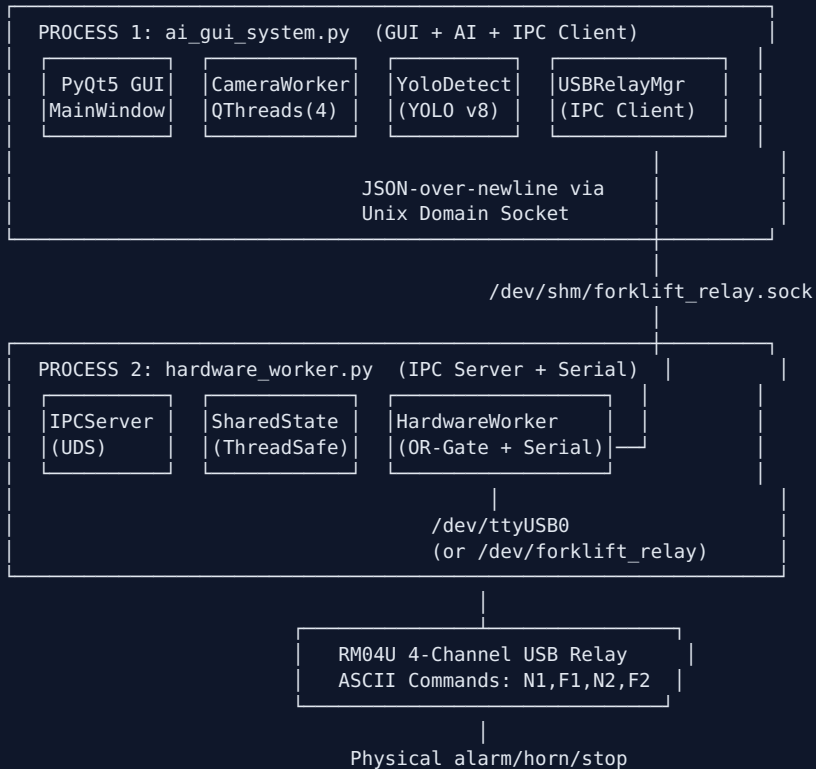
1.9 Software Requirements

Software	Purpose
Ubuntu 24.04 LTS	Operating system (strictly required as documented)
Python 3.10+	Runtime
<code>python3-venv</code>	Virtual environment isolation
<code>libgl1-mesa-glx</code>	OpenCV OpenGL requirement
<code>libxcb-*</code> family	Qt5 XCB platform plugin requirements
X11 Display server	Qt5 GUI (uses <code>QT_QPA_PLATFORM=xcb</code>)
<code>dialout</code> group membership	Serial port access without sudo
udev rule (optional)	Persistent <code>/dev/forklift_relay</code> symlink

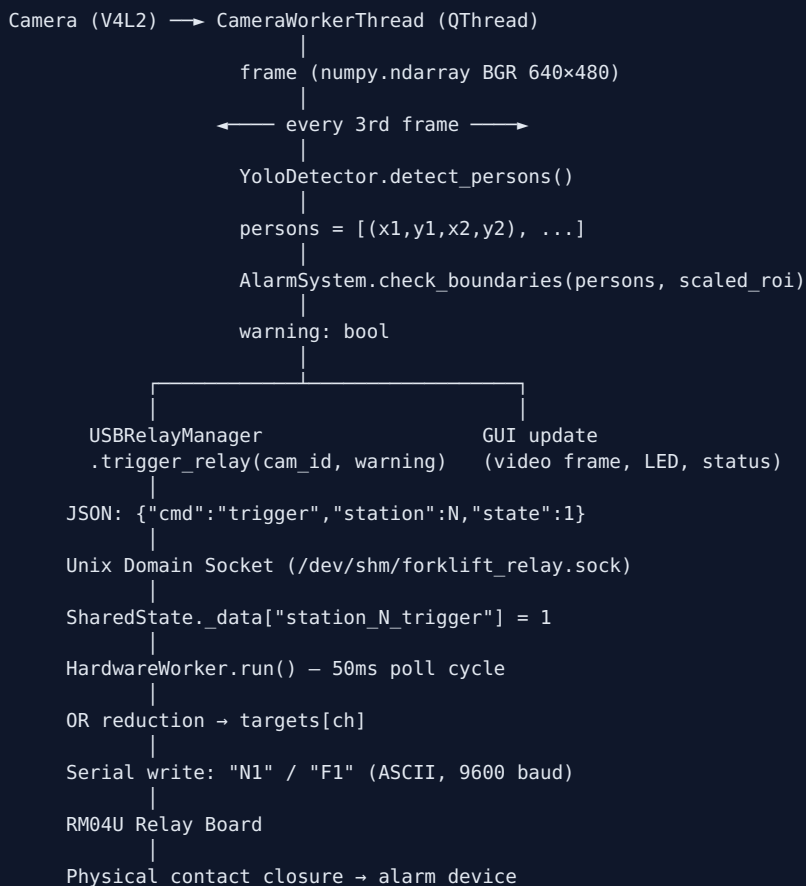
SECTION 2 — PROJECT ARCHITECTURE

2.1 High-Level Overview

The system is split into **two independent OS-level processes** that communicate through a **Unix Domain Socket**:



2.2 Data Flow



2.3 Execution Flow

```

start_system.sh
├─ export DISPLAY, XAUTHORITY
├─ xhost +SI:localuser:$USER
├─ sleep 3 (kernel stabilization)
├─ pkill hardware_worker.py (clean orphans)
├─ rm /dev/shm/forklift_relay.sock (clean stale socket)
├─ source venv/bin/activate
├─ python3 hardware_worker.py & → [BACKGROUND PID saved]
│   └─ check_singleton() (fcntl lock)
│   └─ SharedState() (thread-safe data store)
│   └─ IPCServer().start() → [daemon thread]
│       └─ bind /dev/shm/forklift_relay.sock
│   └─ run_worker_thread() → [daemon thread]
│       └─ get_serial_port()
│       └─ open_serial_connection()
│       └─ run_boot_self_test() (N1,F1,N2,F2,N3,F3,N4,F4)
│       └─ HardwareWorker.run() (50ms OR-gate loop)
├─ poll /dev/shm/forklift_relay.sock every 0.5s (up to 15s)
└─ python3 ai_gui_system.py
    └─ QApplication()
    └─ ForkliftSafetyGUI.__init__()
        └─ SecurityManager()
        └─ load_global_settings()
        └─ auto-detect V4L2 nodes /dev/video0,2,4,6
        └─ YoloDetector(model_path, confidence)
        └─ USBRelayManager() → connect socket IPC
        └─ build full PyQt5 UI
        └─ global_start_all() → 4× CameraWorkerThread.start()
        └─ inactivity_timer.start() (5 min)
        └─ status_check_timer.start() (1 sec)
    └─ app.exec_() [Qt event loop]
  
```

2.4 Thread Flow

Thread	Type	Purpose	Lifecycle
Qt Main Thread	Main	GUI event loop, PyQt5 rendering	Entire application lifetime
<code>CameraWorkerThread</code> × 4	QThread	Camera capture per station	Started on <code>start_camera()</code> , stopped on <code>stop_camera()</code>
<code>USBRelayManager._recv_thread</code>	daemon Thread	Receives IPC socket messages from server	Lives as long as socket is connected
<code>USBRelayManager._connect_socket</code>	daemon Thread	Reconnects socket on disconnect	Short-lived, fires on reconnect need
<code>IPCServer.start</code> (in hw_worker)	daemon Thread	Accepts socket connections	Entire hardware_worker.py lifetime
<code>IPCServer._handle_client</code> per conn	daemon Thread	Handles each GUI client connection	Per-connection lifetime
<code>run_worker_thread</code> (in hw_worker)	daemon Thread	Serial connect + OR-gate loop	Entire hardware_worker.py lifetime

2.5 Communication Flow (IPC Protocol)

All IPC communication is **JSON-over-newline** over a **SOCK_STREAM Unix Domain Socket**.

Client → Server commands:

Command	JSON Structure	Purpose
<code>trigger</code>	<code>{"cmd":"trigger","station":1,"state":1}</code>	Set station alarm state
<code>trigger</code> (manual)	<code>{"cmd":"trigger","station":1,"state":1,"manual":true}</code>	Manual override test
<code>config</code>	<code>{"cmd":"config","hardware_coil_enabled":1}</code>	Update server config
<code>relay_mapping</code>	<code>{"cmd":"relay_mapping","mapping":[1,2,3,4]}</code>	Update relay routing
<code>connect</code>	<code>{"cmd":"connect","boot_self_test":1,...}</code>	Connection handshake
<code>disconnect</code>	<code>{"cmd":"disconnect"}</code>	Reset all triggers
<code>status</code>	<code>{"cmd":"status"}</code>	Request current hw_status

Server → Client responses:

Response	JSON Structure	Purpose
<code>status</code>	<code>{"type":"status","hw_status":"ONLINE"}</code>	Hardware state push
<code>ack</code>	<code>{"type":"ack","cmd":"trigger","success":true}</code>	Command acknowledgement

SECTION 3 — DIRECTORY TREE

```

AI_Forklift_Safety/
├── ai_gui_system.py      ← PRIMARY APPLICATION (1908 lines)
                        GUI + YOLO + IPC Client
                        THIS IS THE MAIN ENTRY POINT FOR OPERATORS
├── hardware_worker.py   ← HARDWARE DAEMON (651 lines)
                        IPC Server + Serial OR-Gate
                        RUNS AS BACKGROUND PROCESS before GUI
├── hardware_manager.py  ← IPC CLIENT CLASS (389 lines)
                        USBRelayManager – socket bridge to hardware_worker
├── detector.py          ← YOLO WRAPPER (49 lines)
                        YoloDetector class used by ai_gui_system.py
├── camera.py            ← CAMERA WRAPPER (53 lines)
                        Simple OpenCV VideoCapture wrapper
├── alarm.py             ← ALARM LOGIC (74 lines)
                        AlarmSystem – ROI boundary violation detection
├── roi.py               ← ROI WIDGET (209 lines)
                        VideoLabel – interactive ROI drawing overlay
├── security.py           ← AUTHENTICATION (57 lines)
                        SecurityManager – 5-role password store
├── osk_widget.py        ← INDUSTRIAL ON-SCREEN TOUCH KEYBOARD (542 lines)
                        Full QWERTY, numeric and symbols virtual keyboard for touch displays
├── config.json           ← RUNTIME CONFIG (48 lines)
                        ROI coordinates + hardware/AI settings
├── security.json         ← PASSWORD STORE (7 lines)
                        Role → plaintext password mapping
├── relay_status.json     ← LEGACY STATE FILE (15 lines)
                        NOT ACTIVELY USED in current IPC architecture
├── relay_statusadmin.json ← LEGACY STATE FILE
                        Same structure as relay_status.json
├── yolov8n.pt           ← YOLO MODEL WEIGHTS (6.5 MB)
                        Pre-trained YOLOv8 nano, COCO dataset
├── requirements.txt      ← CLEAN PYTHON DEPENDENCIES (21 lines)
                        Pure project packages (PyQt5, Ultralytics, Torch, OpenCV, PySerial)
├── .gitignore           ← GIT REPOSITORY HYGIENE
                        Excludes venv/, __pycache__/, *.sock, *.lock, *.log
├── start_system.sh      ← ORCHESTRATOR SCRIPT (78 lines)
                        Includes 15s hardware stabilization delay,
                        --no-delay bypass, worker launch, socket probe, GUI
├── setup_linux.sh       ← DEPLOYMENT SCRIPT (137 lines)
                        One-shot Ubuntu environment setup
├── setup_usb.sh         ← UDEV RULE INSTALLER (6 lines)
                        Creates /dev/forklift_relay symlink
├── test_or_gate_logic.py ← UNIT TEST SUITE (515 lines)
                        Tests hardware_worker.py OR-gate logic (13/13 PASS)
├── AI_Forklift_System.desktop ← LINUX DESKTOP LAUNCHER (11 lines)
                        Double-click launcher for GNOME desktop
├── README.md            ← PROJECT README & GITHUB LANDING (Production Standard)
                        Architecture diagrams, setup guide, test commands, defect logs
└── PROJECT_DOCUMENTATION_AR.md ← ARABIC FULL DOCUMENTATION (670 lines)

```

```

Comprehensive reference guide in Arabic

PROJECT_AUDIT_AND_BUGS_REFERENCE_AR.md ← MASTER AUDIT & DEFECT REGISTRY
Defect catalog, code audit, and full remediation tracking

UBUNTU_MIGRATION_GUIDE.md ← UBUNTU DEPLOYMENT GUIDE
Modern Ubuntu 24.04 LTS deployment reference

hardware.log ← HARDWARE LOG FILE (57 KB)
Runtime log output from hardware_worker.py

industrial_safety_review_report.txt ← SAFETY REVIEW (52 KB)
External industrial safety audit report

systemd & autostart files (System Level / User Session):
├── ~/.config/systemd/user/forklift-ai.service ← Self-healing systemd service
├── ~/.config/autostart/AI_Forklift_Safety.desktop ← Autostart desktop entry
└── ~/Desktop/run_system.sh ← Desktop manual trigger

automated test suites:
├── scratch/test_physical_usb_binding.py ← Physical USB Hub topology binding (9/9 PASS)
├── scratch/test_roi_and_layout_stability.py ← Normalized ROI & Grid stretch tests (5/5 PASS)
├── scratch/test_multi_resolution_adaptation.py ← UI test across 9 screen resolutions (9/9 PASS)
├── scratch/test_card_relay_and_camera_dropdown.py ← Relay dropdown & test button tests (4/4 PASS)
├── scratch/test_production_architecture.py ← IPC & Daemon resilience tests
└── scratch/test_physical_usb_binding.py ← Motherboard USB port binding tests

venv/ ← PYTHON VIRTUAL ENVIRONMENT
└── (Python packages installed here)

forklift_ai_safety/ ← REFACTORED MODULAR SUBPACKAGE
(Earlier architectural version)
NOT USED by ai_gui_system.py
Has its own independent main.py

├── main.py ← Subpackage entry point (28 lines)

├── config/
│   └── security.json ← Subpackage's own security store

├── core/
│   ├── __init__.py
│   ├── camera.py ← Extended camera with SimulatedCamera (176 lines)
│   ├── camera_thread.py ← Integrated detection thread (167 lines)
│   ├── detector.py ← YOLO wrapper without confidence param (42 lines)
│   ├── alarm_manager.py ← Stripped-down alarm (32 lines)
│   ├── roi_manager.py ← File-backed ROI persistence class (64 lines)
│   └── security.py ← 2-role security variant (43 lines)

└── ui/
    ├── __init__.py
    ├── main_window.py ← Alternative main window with scalable grid (310 lines)
    ├── camera_panel.py ← Camera panel widget
    ├── sidebar_panel.py ← Sidebar widget
    ├── status_bar.py ← Status bar widget
    └── video_panel.py ← Video display widget

```

Why `forklift_ai_safety/` exists: This is an earlier architectural refactoring of the same system into a proper Python package structure with MVC separation. It features `SimulatedCamera` mode (no real cameras needed), a scalable N×N grid layout, and more strictly separated UI components. However, it **does not have the hardware relay integration** present in `ai_gui_system.py`. The production system runs `ai_gui_system.py`, not this subpackage.

SECTION 4 — FILE-BY-FILE ANALYSIS

4.1 `ai_gui_system.py`

Size: 1908 lines | **Role:** Primary production application

Purpose: The monolithic main application. It is the entry point for the running system. It creates the Qt window, spawns camera threads, runs YOLO inference, evaluates ROI violations, and communicates with the hardware relay daemon.

Classes Defined: | Class | Purpose | |---|---| | `CircularLED` | Custom SCADA-style LED indicator widget (QWidget subclass) | | `CameraWorkerThread` | Background camera capture thread (QThread subclass) | | `CameraCardWidget` | Complete per-station monitoring panel (QFrame subclass) | | `ForkliftSafetyGUI` | Main application window (QWidget subclass) |

Imports: - Standard: `sys`, `cv2`, `time`, `os`, `json`, `numpy` - PyQt5 Widgets, Core, GUI — full list imported at top - Local: `SecurityManager` (security.py), `Camera` (camera.py), `YoloDetector` (detector.py), `AlarmSystem` (alarm.py), `VideoLabel` (roi.py), `USBRelayManager` (hardware_manager.py)

Dependencies: - Imports 6 local modules (all in same directory) - Requires `yolov8n.pt` at same level or parent directory - Requires `config.json` for settings/ROI persistence - Requires `security.json` for role passwords - Connects to `/dev/shm/forklift_relay.sock` (must be started by `hardware_worker.py` first)

Who calls it: - `start_system.sh` (production launch) - Directly: `python3 ai_gui_system.py`

Inputs: - Camera frames via V4L2 USB cameras - `config.json` — persisted settings and ROI coordinates - `security.json` — role passwords - `/dev/shm/forklift_relay.sock` — hardware status (inbound) and trigger commands (outbound)

Outputs: - PyQt5 fullscreen GUI - JSON writes to `config.json` (ROI saves, settings saves) - Socket commands to `hardware_worker.py` - Console logs (`print()` statements)

Global Variables / Constants: None at module level (all logic is class-encapsulated)

Configuration: - `load_global_settings()` reads `config.json["settings"]` - Default: `com_port=/dev/ttyUSB0`, `baud_rate=9600`, `model_path=yolov8n.pt`, `confidence=0.25`, `on_delay=0.2`, `off_delay=1.5`

Potential Problems: - `USBRelayManager(baud_rate=9600)` on line 813 passes a `baud_rate` kwarg, but `USBRelayManager.__init__` (in `hardware_manager.py`) does not accept it — this is a silent interface mismatch (the kwarg is ignored if Python allows extra kwargs, but will raise a `TypeError` if it does not) - Config path `"config.json"` is a relative path — must be run from the project directory - `relay_manager.get_hw_status()` is called on line 1822, but `USBRelayManager` exposes `hw_status` as a **property**, not a method — this will raise `TypeError: 'str' object is not callable`. The correct call is `self.relay_manager.hw_status`

Future Improvements: - Split into proper MVC modules (similar to `forklift_ai_safety/` package) - Replace relative paths with `pathlib.Path(__file__).parent` - Add proper logging with `logging` module instead of `print()`

4.2 `hardware_worker.py`

Size: 651 lines | **Role:** Hardware daemon / IPC server

Purpose: Runs as a **standalone background OS process** (not a thread). Owns the serial port connection to the RM04U relay board. Implements the pure OR-gate logic that maps station trigger states to relay channel outputs. Communicates with the GUI process over a Unix Domain Socket.

Classes Defined: | Class | Purpose | |---|---| | `SharedState` | Thread-safe data store for all IPC state (station triggers, relay mapping, config) | | `IPCServer` | Unix Domain Socket server; accepts connections from GUI, processes commands | | `HardwareWorker` | Pure OR-gate consumer; polls `SharedState` every 50ms, writes serial commands |

Functions at Module Level: | Function | Purpose | |---|---| | `get_config_port()` | Reads `com_port` from config.json with retry | | `get_serial_port()` | Priority-ordered port resolution: udev > config > glob fallback | | `is_port_present()` | Checks if a serial device file exists | | `open_serial_connection()` | Opens pyserial connection with CH340 stabilization delay | | `load_initial_config()` | Loads config.json settings for initial SharedState seeding | | `run_boot_self_test()` | Sequential N1/F1 through N4/F4 relay cycling | | `run_worker_thread()` | Outer reconnect loop: serial open → self-test → OR-gate | | `check_singleton()` | fcntl file-lock prevents duplicate daemon instances | | `main()` | Entry point: creates SharedState, IPCServer, worker thread |

Configuration Constants:

```

SOCKET_PATH      = "/dev/shm/forklift_relay.sock"  # IPC endpoint
LOCK_PATH        = "/tmp/hardware_worker.lock"    # Singleton guard
UDEV_RELAY_PATH  = "/dev/forklift_relay"          # udev persistent symlink
BAUD_RATE        = 9600                          # Serial baud rate
SERIAL_TIMEOUT   = 1                             # Serial read timeout (seconds)
POLL_INTERVAL    = 0.05                          # OR-gate cycle (50ms)
RECONNECT_INTERVAL = 3.0                         # Seconds between reconnect attempts
SELF_TEST_PULSE  = 0.3                           # Seconds each relay stays ON in self-test

```

Dependencies: - `pyserial` (must be installed; process exits if missing) - `config.json` (optional; uses defaults if missing) - Filesystem: `/dev/shm/` (tmpfs RAM disk) - Filesystem: `/dev/forklift_relay` or `/dev/ttyUSB*`

Who calls it: `start_system.sh` via `python3 hardware_worker.py &`

Potential Problems: - If `/dev/shm/` does not exist (non-Linux systems), the socket path will fail - The singleton lock at `/tmp/hardware_worker.lock` is a file lock; if the process crashes without cleanup, the lock file may remain but will not block re-entry (flock releases on process death) - `load_initial_config()` initializes defaults that include `on_delay` and `off_delay`, but these are intentionally NOT stored in `SharedState.data` — this is correct by design but can confuse engineers who expect them there

4.3 `hardware_manager.py`

Size: 389 lines | **Role:** IPC client library

Purpose: Provides the `USBRelayManager` class — the GUI-side interface to the hardware system. The GUI calls methods on this object as if it were directly controlling hardware; internally, every call translates to a JSON IPC command over the Unix Domain Socket to `hardware_worker.py`.

Classes Defined: | Class | Purpose | |---|---| | `USBRelayManager` | Full-featured IPC client with auto-reconnect, command queuing, hardware status caching |

Key Attributes:

```

self.is_connected      # bool: GUI considers relay "connected"
self.relay_states      # [boolx4]: per-channel last-sent state latch
self.last_cmd_time     # [floatx4]: timestamp of last command per channel
self.on_delay          # float: debounce delay (client-side only)
self.off_delay         # float: hold delay (client-side only)
self.hw_status         # str: cached "ONLINE"/"OFFLINE" from server push
self.manual_overrides  # [boolx4]: manual test override flags (NOTE: not in __init__, added dynamically)

```

Notable Design Decision: `manual_overrides` is not initialized in `__init__` — it is accessed via `self.relay_manager.manual_overrides[idx]` in `ai_gui_system.py`. This means it must be set externally before first use or an `AttributeError` will occur. It is set in `show_live_page()` but may not be set before the first frame arrives.

Dependencies: `socket`, `threading`, `json`, `time`, `glob`, optionally `serial.tools.list_ports`

4.4 `detector.py`

Size: 49 lines | **Role:** YOLO inference wrapper

Purpose: Wraps the Ultralytics YOLO API. Loads the model at construction and provides `detect_persons()` which runs inference and returns only COCO class 0 (person) bounding boxes.

Key Design Decisions: - Looks for model in parent directory first (`../yolov8n.pt`), then current directory — handles running from either `forklift_ai_safety/` or the project root - `imgsz=320` default (overridable): deliberately half the standard 640 to reduce CPU load - `verbose=False` suppresses YOLO's console output on every frame

4.5 `camera.py`

Size: 53 lines | **Role:** OpenCV camera wrapper

Purpose: Thin wrapper around `cv2.VideoCapture`. Sets resolution properties on open. Provides `read_frame()` and `release()`.

Key Design: - Catches all exceptions during init silently — sets `self.cap = None` - `read_frame()` returns `(bool, frame_or_None)` tuple — matches OpenCV convention - Does **not** handle reconnection — reconnection logic is in `CameraWorkerThread`

4.6 `alarm.py`

Size: 74 lines | **Role:** Safety logic evaluation

Purpose: Implements the core safety decision: given a list of detected person bounding boxes and an ROI rectangle, determines if any person is inside the hazard zone.

Detection Method: Center-point containment — uses the center `(x1+x2)/2, (y1+y2)/2` of each bounding box and checks if it falls within the ROI using `QRect.contains(QPoint)`.

Important Assumption: A person is only detected as "in zone" if their **body center** is inside the ROI. A person standing at the ROI edge with their center just outside will NOT trigger an alarm. This may be a safety concern for very wide people or partial occlusions.

Methods: | Method | Description | |---|---| | `evaluate_safety(persons, roi_rect)` | Main logic: returns True if any person center is in ROI | | `check_boundaries(persons, roi_rect)` | Alias for `evaluate_safety()` (backward compatibility) | | `get_status_styles(warning)` | Returns (text, css_string) for GUI label — not used in main system | | `get_camera_error_styles()` | Returns error text/CSS — not used in main system |

4.7 `roi.py`

Size: 209 lines | **Role:** Interactive ROI drawing widget

Purpose: Custom `QLabel` subclass that overlays an interactive Region of Interest rectangle on top of the camera video frame. Supports draw, move, and resize operations via mouse events. Emits a `roi_updated` signal when the ROI changes.

State Machine:

```

States: {is_drawing, dragging, resizing}
Transitions:
  MousePress on handle → resizing = True
  MousePress inside ROI → dragging = True
  MousePress elsewhere (drawing_roi=True) → is_drawing = True
  MouseRelease → all states False, emit roi_updated

```

Handle System: 4 corner handles (12×12 pixel squares) for resize. Cursor changes to diagonal arrows over handles, move cursor inside ROI, arrow elsewhere.

Coordinate System: ROI coordinates are in **widget pixel space** (the display coordinates on-screen). When used for detection, they are scaled to frame coordinates in `on_frame_received()` in `CameraCardWidget`.

4.8 `security.py`

Size: 57 lines | **Role:** Authentication manager

Purpose: Manages the 5-role access control system. Reads/writes `security.json` which stores plaintext passwords mapped by role name.

5 Roles: | Role | Default Password | Access Level | |---|---|---| | Operator | ` (empty) | 1 – View only, cannot close app | | Technician | tech | 2 – (reserved, no specific UI gates) | | Supervisor | 1111 | 3 – Full UI access, close app | | Engineer | admin | 4 – Full access | | Administrator | superadmin` | 5 — Full access |

Security Warning: Passwords are stored and compared in **plaintext**. There is no hashing, salting, or encryption.

4.9 `config.json`

Type: JSON Configuration File | **Size:** 48 lines

Purpose: Single-file store for all persisted runtime settings and per-camera ROI coordinates.

Structure:

```

{
  "camera_0": {"x": 0, "y": 184, "w": 723, "h": 144},
  "camera_1": {"x": 157, "y": 34, "w": 187, "h": 163},
  "camera_2": {"x": 44, "y": 54, "w": 254, "h": 259},
  "camera_3": {"x": 103, "y": 60, "w": 237, "h": 252},
  "settings": {
    "com_port": "/dev/ttyUSB0",
    "baud_rate": 9600,
    "camera_mapping": ["/dev/video0", "/dev/video2", "/dev/video4", 6],
    "relay_mapping": [3, 2, 3, 2],
    "model_path": "yolov8n.pt",
    "confidence": 0.35,
    "on_delay": 0.0,
    "off_delay": 0.1,
    "boot_self_test": 1,
    "hardware_coil_enabled": 1
  }
}

```

Field Descriptions:

Key	Type	Description
<code>camera_0</code> ... <code>camera_3</code>	dict	ROI in widget pixels: x, y, w, h
<code>settings.com_port</code>	string	Serial port for relay board
<code>settings.baud_rate</code>	int	Serial baud rate (default 9600)
<code>settings.camera_mapping</code>	array	Device path or index for each station
<code>settings.relay_mapping</code>	array[4]	Which relay channel each station triggers
<code>settings.model_path</code>	string	YOLO weights filename
<code>settings.confidence</code>	float	YOLO detection threshold (0.0–1.0)
<code>settings.on_delay</code>	float	Client-side debounce delay (seconds)
<code>settings.off_delay</code>	float	Client-side hold delay after zone clear
<code>settings.boot_self_test</code>	0/1	Run relay self-test on daemon startup
<code>settings.hardware_coil_enabled</code>	0/1	Global hardware output enable/disable

4.10 `security.json`

Type: JSON Password Store | **Size:** 7 lines

Warning: Contains plaintext passwords for all roles. File permissions should be restricted (`chmod 600 security.json`). Currently world-readable.

4.11 `start_system.sh`

Type: Bash orchestrator script | **Size:** 64 lines

Purpose: Correctly sequences the startup of both processes, ensures the hardware socket exists before launching the GUI, and handles cleanup on exit.

Execution Steps: 1. Set `DISPLAY` and `XAUTHORITY` environment variables 2. `xhost` grant for local user X11 access 3. `sleep 3` — wait for kernel USB enumeration 4. `pkill hardware_worker.py` — kill any orphaned daemons 5. `rm -f /dev/shm/forklift_relay.sock` — clean stale socket 6. `source venv/bin/activate` — activate Python environment 7. `python3 hardware_worker.py &` — launch daemon in background 8. Poll `/dev/shm/forklift_relay.sock` every 0.5s (up to 15 seconds) 9. `python3 ai_gui_system.py` — launch GUI (blocks until GUI exits) 10. `kill $HW_PID` — terminate daemon 11. `rm -f /dev/shm/forklift_relay.sock` — clean socket on exit

Critical Note: The socket polling loop waits up to 15 seconds (30 iterations × 0.5s). The socket is created by `hardware_worker.py` after: CH340 stabilization (2s) + self-test (4ch × 2 × 0.3s = 2.4s) ≈ 4.4s minimum. The 15s window gives ample margin.

4.12 `setup_linux.sh`

Type: Deployment script | **Size:** 137 lines

Purpose: One-shot Ubuntu environment setup for a fresh machine. Installs system graphics libraries, Python, creates venv, installs Python packages from `requirements.txt`, and adds user to `dialout` group.

4.13 `setup_usb.sh`

Type: udev rule installer | **Size:** 6 lines

Purpose: Creates a permanent `/dev/forklift_relay` symlink for the RM04U relay board (CH340 chip, VID:1a86, PID: 7523). This ensures the relay always gets the same device name regardless of USB port used.

udev Rule Created:

```
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", SYMLINK+="forklift_relay", MODE="0666"
```

Written to: `/etc/udev/rules.d/99-forklift-relay.rules`

4.14 `test_or_gate_logic.py`

Type: Unit test suite | **Size:** 515 lines

Purpose: Verifies the OR-gate logic in `hardware_worker.py` without needing physical hardware. Uses `FakeSerial` to capture written serial commands and `FakeIPCTServer` to absorb status broadcasts.

Key Design: Uses `importlib.util.spec_from_file_location` to import `hardware_worker.py` as a module, then patches `serial` in `sys.modules` with a stub before import — so pyserial is not required to run the tests.

4.15 `relay_status.json` / `relay_statusadmin.json`

Type: Legacy state files | **Size:** 340 bytes each

Status: These files are **NOT actively used** in the current IPC Socket architecture. They appear to be artifacts from an earlier version where state was shared via file polling instead of Unix Domain Sockets. They are safe to ignore but should not be deleted until confirmed obsolete.

4.16 `AI_Forklift_System.desktop`

Type: Linux desktop launcher | **Size:** 240 bytes

Purpose: Allows users to launch the system by double-clicking an icon on the Ubuntu desktop.

Note: The `Exec` path is hardcoded to `/home/limitlessfuture/Desktop/run_system.sh`. This will **not work** on the current user's system if the username is different (current user is `forklift`). This file requires updating to match the actual username.

4.17 `forklift_ai_safety/` Subpackage Files

These files form an **alternate refactored version** of the application with better code organization but without the production hardware integration.

File	Key Difference from Production
<code>forklift_ai_safety/main.py</code>	Uses <code>ui/main_window.py</code> instead of <code>ForkliftSafetyGUI</code>
<code>forklift_ai_safety/core/camera.py</code>	Adds <code>SimulatedCamera</code> class for testing without hardware
<code>forklift_ai_safety/core/camera_thread.py</code>	All-in-one thread: capture + YOLO + alarm check + render
<code>forklift_ai_safety/core/detector.py</code>	No <code>confidence</code> or <code>imgsz</code> parameters
<code>forklift_ai_safety/core/alarm_manager.py</code>	Stripped version; no <code>get_status_styles()</code>
<code>forklift_ai_safety/core/roi_manager.py</code>	Separate JSON file (<code>roi.json</code>); contains a stray <code>stream = None</code> line at EOF (bug)
<code>forklift_ai_safety/core/security.py</code>	Only 2 roles (Supervisor, Engineer); stores in <code>config/security.json</code>
<code>forklift_ai_safety/ui/main_window.py</code>	Scalable N×N grid; no hardware relay integration
<code>forklift_ai_safety/config/security.json</code>	2-role password store with default passwords <code>1234</code> and <code>admin</code>

SECTION 5 — CLASS ANALYSIS

5.1 `CircularLED` (`ai_gui_system.py`)

Purpose: Custom-drawn circular indicator widget mimicking physical SCADA panel LEDs with glass-reflection gradients.

Attributes: | Attribute | Type | Description | |---|---|---| | `state` | str | Current state: `"standby"`, `"secure"`, `"breach"`, `"error"` | | `flash_state` | bool | Current flash phase for breach animation | | `flash_timer` | QTimer | 500ms timer for breach flashing |

Constructor: `__init__(parent=None)` — Sets fixed 14×14 pixel size, initializes state to `"standby"`, creates flash timer.

Methods: | Method | Description | |---|---| | `set_state(state)` | Changes LED state, starts/stops flash timer | | `toggle_flash()` | Inverts `flash_state`, triggers repaint | | `paintEvent(event)` | Draws radial gradient ellipse with state-dependent colors |

Colors by State: - `standby` → gray (`#646e78` bright, `#323738` dark) - `secure` → green (`#2ecc71` bright, `#27ae60` dark) - `breach` → red, flashing (`#ff6464` / `#e74c3c`) - `error` → amber (`#f39c12` bright, `#d35400` dark)

Lifecycle: Created once per `CameraCardWidget`. Destroyed when camera card is destroyed.

5.2 `CameraWorkerThread` (`ai_gui_system.py`)

Purpose: Background QThread that continuously reads frames from one camera and emits them to the GUI thread.

Attributes: | Attribute | Type | Description | |---|---|---| | `device_index` | int or str | Camera device index or path | | `width`, `height` | int | Resolution (default 640×480) | | `running` | bool | Loop control flag | | `camera` | Camera | Camera instance (or None if offline) |

Signals: | Signal | Parameters | Description | |---|---|---| | `frame_ready` | `np.ndarray` | Emitted each time a valid frame is captured | | `status_changed` | `(bool, str)` | Emitted when camera goes online/offline |

Reconnection Logic: If `camera.cap` is None or not opened, waits 3 seconds then attempts to create a new `Camera` instance. Emits `status_changed(False, "Offline")` during reconnection attempts.

Frame Rate: `time.sleep(0.03)` after each successful frame = target ~33 FPS raw capture rate.

Lifecycle: Created in `start_camera()`, stopped in `stop_camera()`. `stop()` calls `self.wait(3000)` — waits up to 3 seconds for thread to exit.

5.3 `CameraCardWidget` (`ai_gui_system.py`)

Purpose: The most complex class. Encapsulates the complete per-station UI card: video display, YOLO detection, ROI evaluation, hardware trigger dispatch, and all local state.

Attributes: | Attribute | Type | Description | |---|---|---| | `cam_id` | int | Station index 0–3 | | `device_index` | int/str | V4L2 device node | | `detector` | YoloDetector | Shared YOLO model instance | | `security` | SecurityManager | Shared security instance | | `relay_manager` | USBRelayManager | Shared hardware IPC client | | `camera_online` | bool | True when camera is streaming | | `camera_thread` | CameraWorkerThread | Background thread | | `alarm_system` | AlarmSystem | Per-station alarm evaluator | | `prev_warning` | bool | Last warning state (for relay transition) | | `last_hardware_state` | bool/None | Last state written to hardware (edge detection gate) | | `frame_counter` | int | Used for frame-skip throttling | | `was_overridden` | bool | True when manual override was active |

Key Methods: | Method | Description | |---|---| | `load_roi()` | Reads ROI from config.json for this `cam_id` | | `save_roi(roi_rect)` | Writes ROI to config.json | | `on_roi_updated(roi_rect)` | Called by VideoLabel signal; saves if editable | | `start_camera()` | Spawns CameraWorkerThread and blink timer | | `stop_camera()` | Stops thread and timer, resets hardware relay if was in breach | | `toggle_roi_edit()` | Toggles ROI drawing mode on/off | | `on_frame_received(frame)` | **Critical path:** runs inference, alarm check, hardware dispatch, UI update | | `update_standby_frame()` | Renders dark standby image to video label | | `update_offline_banner()` | Renders flashing "OFFLINE" banner |

Frame Processing in `on_frame_received()`: 1. Calculate scale factors (widget pixels → frame pixels) 2. Scale ROI coordinates to frame coordinates 3. Frame skip: run YOLO only every 3rd frame (`frame_counter % 3 == 0`) 4. Call `detector.detect_persons(frame, imgsz=320)` 5. Call `alarm_system.check_boundaries(persons, scaled_roi)` 6. Check `relay_manager.manual_overrides[cam_id]` — skip hardware if manual override active 7. Edge-detect `warning != last_hardware_state` → only write to relay on state change 8. Call `relay_manager.trigger_relay(cam_id, warning)` 9. Draw bounding boxes, ROI overlay, alert text on frame 10. Convert BGR frame to RGB QImage and display 11. Call `parent_window.evaluate_global_alarms()`

5.4 `ForkliftSafetyGUI` (`ai_gui_system.py`)

Purpose: Main application window. Owns the global state, all 4 camera cards, the sidebar, the stacked widget navigation, and system-wide controls.

Attributes: | Attribute | Type | Description | |---|---|---| | `current_role` | str | Active role (default "Operator") | | `access_levels` | dict | Role → integer level mapping | | `security` | SecurityManager | Authentication handler | | `global_settings` | dict | Loaded/saved application configuration | | `detector` | YoloDetector | Shared YOLO model (passed to all cards) | | `relay_manager` | USBRelayManager | Shared hardware IPC client | | `cards` | list[CameraCardWidget] | 4 camera station cards | | `inactivity_timer` | QTimer | 5-minute auto-logout | | `status_check_timer` | QTimer | 1-second hardware status poll | | `hardware_is_online` | bool | Cached relay module online state |

Pages (QStackedWidget): - **Page 0** (`live_view_page`): 2×2 grid of CameraCardWidget instances — the main monitoring view - **Page 1** (`setup_page`): System configuration panel (relay, camera mapping, AI config, timers)

Key Methods: | Method | Description | |---|---| | `eventFilter()` | Resets inactivity timer on any user interaction | | `auto_logout()` | Reverts role to Operator on inactivity | | `show_live_page()` | Clears manual overrides, switches to page 0 | | `show_config_page()` | Gated by Supervisor access; switches to page 1 | | `load_global_settings()` | Reads config.json settings into `global_settings` dict | | `save_global_configurations()` | Writes all config page values to config.json and applies live | | `instant_save_timer_config()` | Auto-saves timer/relay settings on any widget change | | `evaluate_global_alarms()` | Aggregates warning states from all cards, updates footer banner | | `poll_hardware_status()` | Reads cached hw_status, updates blink state and footer | | `change_role(role)` | Prompts password, upgrades access level | | `update_ui_permissions()` | Enables/disables UI controls based on access level | | `change_password()` | Gated password change dialog | | `closeEvent(event)` | Blocks close if level < 3; executes teardown if authorized | | `global_start_all()` | Calls `start_camera()` on all 4 cards | | `global_stop_all()` | Requires Supervisor; calls `stop_camera()` on all 4 cards | | `pause_all_card_timers()` | Stops all CameraWorkerThreads (used during serial reconnect) | | `resume_all_card_timers()` | Restarts all previously-active CameraWorkerThreads |

5.5 SharedState (hardware_worker.py)

Purpose: Thread-safe data store shared between the `IPCServer` (writer) and `HardwareWorker` (reader) threads inside the hardware daemon process.

Internal Data Structure (`_data` dict):

```
{
    "station_1_trigger": 0,      # 0 or 1 – set by GUI trigger commands
    "station_2_trigger": 0,
    "station_3_trigger": 0,
    "station_4_trigger": 0,
    "station_1_relay": 1,       # Relay channel for station 1 (1–4)
    "station_2_relay": 2,
    "station_3_relay": 3,
    "station_4_relay": 4,
    "manual_test": 0,           # 1 = manual test active (overrides coil_enabled=0)
    "boot_self_test": 1,        # 1 = run self-test on boot
    "hardware_coil_enabled": 1,  # 0 = suppress all relay outputs
    "hw_status": "OFFLINE",     # "ONLINE" or "OFFLINE"
}
```

Synchronization: All read and write access is guarded by `self._lock = threading.Lock()`. `get_snapshot()` returns a full dictionary copy under the lock — allowing callers to read all values atomically without the lock held during processing.

Methods: | Method | Description | |---|---| | `get_snapshot()` | Returns full dict copy (atomic read) | | `get(key, default)` | Single-key read under lock | | `set(key, value)` | Single-key write under lock | | `update(updates)` | Multi-key dict update under lock | | `set_hw_status(status_str)` | Sets hw_status; returns True only if value changed | | `reset_triggers()` | Resets all station triggers and manual_test to 0 |

5.6 IPCServer (hardware_worker.py)

Purpose: Unix Domain Socket server. Accepts client connections from the GUI process, receives JSON commands, updates SharedState, and broadcasts status to all connected clients.

Attributes: | Attribute | Type | Description | |---|---|---| | `socket_path` | str | Path to the Unix socket file | | `state` | SharedState | Reference to shared state | | `clients` | list | Connected client sockets | | `clients_lock` | threading.Lock |

Guards access to `clients` list || `server_socket` | socket | The listening server socket || `running` | bool | Loop control flag |

Per-Client Handler `_handle_client()` : Runs in its own daemon thread per connection. Buffers incoming data, splits on `\n`, and calls `_process_command()` for each complete JSON line.

Command Handlers: | Command | Handler Action | `trigger` | Sets `station_N_trigger` in `SharedState` || `manual_test` | Sets `manual_test` flag || `config` | Updates `hardware_coil_enabled`, `boot_self_test` in `SharedState` || `relay_mapping` | Updates `station_N_relay` values in `SharedState` || `connect` | Resets triggers, applies new config, sends current `hw_status` || `disconnect` | Resets all triggers || `status` | Sends current `hw_status` to requesting client |

`broadcast_status()` : Sends current `hw_status` to all connected clients. Removes dead clients detected by send failures.

5.7 `HardwareWorker` (`hardware_worker.py`)

Purpose: The core OR-gate logic. A stateless poll loop that reads station states from `SharedState`, computes the logical OR for each relay channel, and transmits ASCII serial commands **only when the result changes**.

Attributes: | Attribute | Type | Description | `ser` | serial.Serial | Open serial port handle || `state` | `SharedState` | Shared state reference || `ipc` | IPCServer | For broadcasting status changes || `last_sent_state` | dict {1–4: None/0/1} | Per-relay last transmitted state |

The 3-Phase OR-Gate Cycle (runs every 50ms):

Phase 1 — Station Registry:

```
for station in range(1, 5):
    raw = config.get(f"station_{station}_trigger", 0)
    station_states[station] = 1 if int(raw) == 1 else 0
```

Reads all 4 station triggers as strict 0/1 integers.

Phase 2 — OR Reduction:

```
targets = {1: 0, 2: 0, 3: 0, 4: 0}
if coil_enabled or is_manual:
    for relay_ch in range(1, 5):
        for station in range(1, 5):
            mapped_relay = int(config.get(f"station_{station}_relay", station))
            if mapped_relay == relay_ch and station_states[station] == 1:
                targets[relay_ch] = 1
                break # OR satisfied
```

Computes target state for each physical relay channel.

Phase 3 — Edge-Triggered Serial Output:

```
for ch in range(1, 5):
    if targets[ch] != last_sent_state[ch]:
        self.transmit(ch, bool(targets[ch]))
```

Only transmits when the state changes from last sent.

`transmit(channel, state_on)` : Sends `"N{ch}"` (ON) or `"F{ch}"` (OFF) as ASCII bytes via `ser.write()` + `ser.flush()`. Updates `last_sent_state[channel]`.

Exception Handling: - `KeyboardInterrupt` → clean shutdown: turns off all ON relays before returning - `serial.SerialException` → marks hardware OFFLINE, returns False (triggers reconnect) - General `Exception` → logs warning, sleeps, continues

5.8 `USBRelayManager` (`hardware_manager.py`)

Purpose: The GUI-side IPC client. Provides a high-level hardware control API while managing the Unix Domain Socket connection in the background.

Key Design: `trigger_relay(camera_id, state, force=False)`

This is the most critical method. Its complete flow: 1. Convert `camera_id` (0-based) to `channel_index` (1-based) 2. Validate channel range (1–4) 3. **State-change latch:** if `not force` and `relay_states[ch-1] == state` → return early (no duplicate sends) 4. **Cooldown guard (ON only):** if less than 500ms since last ON command → return early (prevents rapid-fire chatter). OFF commands always pass through immediately. 5. Update `relay_states` and `last_cmd_time` 6. Send `{"cmd": "trigger", "station": N, "state": 0/1}` via socket 7. Return `(True, description)` or `(False, error)`

Auto-Reconnect: The `_receiver_loop()` background thread detects connection loss (zero-length recv, broken pipe). On loss: closes socket, sets `_hw_status = "OFFLINE"`, sleeps 2 seconds, calls `_connect_socket()` to reconnect. This loop runs until `_recv_running = False`.

5.9 `SecurityManager` (`security.py`)

Purpose: Load, verify, and update role passwords stored in `security.json`.

Key Behavior: - On init, creates `security.json` with default passwords if it doesn't exist - Also upgrades existing file to add any missing roles - `verify_password(role, password)` does a simple string equality check — no hashing - `change_password(role, new_password)` overwrites the file with updated passwords

5.10 `AlarmSystem` (`alarm.py`)

Purpose: Evaluate whether any detected person is inside the ROI.

Algorithm:

```
for (x1, y1, x2, y2) in persons:
    center = QPoint((x1+x2)//2, (y1+y2)//2)
    if roi_rect.contains(center):
        return True
return False
```

Return: `True` = violation (relay should fire), `False` = safe.

5.11 `VideoLabel` (`roi.py`)

Purpose: Interactive QLabel that overlays an ROI rectangle on video frames and allows drag-draw-resize via mouse.

State Flags: | Flag | Meaning | `is_editable` | Master permission flag (set by role level) | `drawing_roi` | Draw-new-rectangle mode active | `is_drawing` | Currently drawing (mouse held) | `dragging` | Currently moving ROI | `resizing` | Currently resizing via corner handle |

SECTION 6 — FUNCTION ANALYSIS

6.1 `get_serial_port()` — `hardware_worker.py`

Purpose: Determines which serial port path to use, in priority order.

Priority: 1. `/dev/forklift_relay` — udev persistent symlink 2. `config.json` `com_port` setting (if it doesn't start with `COM` and file exists) 3. First `/dev/ttyUSB*` or `/dev/ttyACM*` found 4. Fallback: `/dev/ttyUSB0`

Parameters: None **Returns:** str — Serial port path **Exceptions:** None (all exceptions absorbed, returns fallback)

6.2 `open_serial_connection(port_name)` — `hardware_worker.py`

Purpose: Opens a pyserial connection and waits for CH340 firmware to stabilize.

Parameters: `port_name` (str) **Returns:** `serial.Serial` object or `None` **Critical Detail:** `time.sleep(2)` — The CH340 USB-Serial chip requires 2 seconds after port open before it accepts commands. Removing this causes silent write failures.

6.3 `run_boot_self_test(ser)` — `hardware_worker.py`

Purpose: Validates all 4 relay channels by toggling each one ON then OFF with a 0.3s pause.

Algorithm:

```
for ch in 1..4:
    write "Nch" → flush → sleep(0.3)
    write "Fch" → flush → sleep(0.3)
```

Returns: `True` if all 4 channels succeed, `False` on any exception. **Side Effect:** Each relay physically clicks ON then OFF — audible/visible during boot.

6.4 `run_worker_thread(shared_state, ipc_server)` — `hardware_worker.py`

Purpose: Outer reconnect loop. Keeps trying to open serial and run the OR-gate until a clean exit.

Algorithm:

```
while True:
    port = get_serial_port()
    ser = open_serial_connection(port)
    if ser is None: sleep(3); continue
    if boot_self_test: run_boot_self_test(ser); if fails: sleep(3); continue
    set_hw_status("ONLINE"); broadcast
    worker = HardwareWorker(ser, state, ipc)
    clean_exit = worker.run()
    set_hw_status("OFFLINE"); broadcast
    close ser
    if clean_exit: break
    sleep(3)
```

6.5 `check_singleton()` — `hardware_worker.py`

Purpose: Prevents two instances of `hardware_worker.py` from running simultaneously (which would cause serial port conflict).

Implementation: Opens `/tmp/hardware_worker.lock` and applies `fcntl.LOCK_EX | LOCK_NB` (exclusive non-blocking). If another process holds the lock, `IOError` is raised and the process exits.

Note: Linux `fcntl` locks are automatically released when the process dies, so no cleanup needed.

6.6 `CameraCardWidget.on_frame_received(frame)` — `ai_gui_system.py`

Purpose: The main real-time processing pipeline. Called by `frame_ready` signal from `CameraWorkerThread` for every captured frame.

Full Algorithm: 1. Extract frame dimensions 2. Compute `scale_x = frame_w / widget_w`, `scale_y = frame_h / widget_h` 3. Scale ROI from widget coordinates to frame coordinates 4. Increment `frame_counter`; decide `run_inference = (frame_counter % 3 == 0)` 5. If `run_inference`: call `detector.detect_persons(frame, imgsiz=320)`, call `alarm_system.check_boundaries(persons, scaled_roi)`, cache results 6. If not `run_inference`: reuse `last_persons`, `last_warning`, `last_scaled_roi` 7. Update `metric_objects_label` 8. Draw bounding boxes on frame (blue=danger, red=safe — **note: counterintuitive color assignment**) 9. Draw ROI overlay on frame (red=danger, green=safe) 10. Check `relay_manager.manual_overrides[cam_id]`; if override active, skip hardware 11. If not override: if `warning != last_hardware_state` → call `trigger_relay(cam_id, warning)`; update `last_hardware_state` 12. Update all LED, banner, title, card style based on `warning` 13. Calculate FPS, draw on frame 14. Convert BGR → RGB, wrap in QImage, set as pixmap 15. Call `parent_window.evaluate_global_alarms()`

Performance: Called at up to 33 FPS. YOLO runs at ~11 FPS effective (every 3rd frame). GUI update runs at full 33 FPS.

6.7 `ForkliftSafetyGUI.evaluate_global_alarms()` — `ai_gui_system.py`

Purpose: Aggregates warning states across all 4 cards and updates the footer status banner.

Logic:

```
any_warning = any card has property("warning") == "true"
hardware_online = self.hardware_is_online

if not hardware_online:
    footer = RED blinking "RELAY MODULE: DISCONNECTED"
elif any_warning:
    footer = RED "CRITICAL WARNING: PATH VIOLATION"
else:
    footer = GREEN "RELAY MODULE: CONNECTED"
```

6.8 `HardwareWorker.transmit(channel, state_on)` — `hardware_worker.py`

Purpose: Writes a single relay command to the serial port.

Parameters: - `channel` (int 1–4): Physical relay channel - `state_on` (bool): True=ON, False=OFF

Serial Protocol: ASCII string: `"N{ch}"` for ON, `"F{ch}"` for OFF. Example: `"N1"` = Relay 1 ON, `"F3"` = Relay 3 OFF. No checksum, no acknowledgement expected from relay board.

Raises: `serial.SerialException` if serial port is not open.

6.9 ForkLiftSafetyGUI.change_role(role) — ai_gui_system.py

Purpose: Handles role changes from the sidebar combobox. Prompts for password, verifies, updates access level.

Special Case: Changing to "Operator" requires no password — always allowed.

On Failure: Silently reverts the combobox to the previous role (using `blockSignals(True/False)` to prevent recursive signal).

6.10 ForkLiftSafetyGUI.closeEvent(event) — ai_gui_system.py

Purpose: Intercepts window close events (including Alt+F4 and X button).

Logic: - If role level < 3 (Supervisor): `event.ignore()` — blocks the close - If role level ≥ 3: stops all cameras, disconnects relay, calls `relay_manager.cleanup()`, calls `event.accept()`

SECTION 7 — LINE-BY-LINE EXPLANATION

7.1 hardware_worker.py — SharedState Initialization (lines 144–194)

```
class SharedState:
    def __init__(self):
        self._lock = threading.Lock()
        config = load_initial_config()
        mapping = config.get("relay_mapping", [1, 2, 3, 4])
        while len(mapping) < 4:
            mapping.append(len(mapping) + 1)
```

- `self._lock`: A mutex that every method must acquire before reading or writing `_data`. Without this, concurrent reads and writes from the IPC server thread and the OR-gate worker thread would cause race conditions (e.g., reading a half-updated dict).
- `load_initial_config()`: Reads `config.json` to get initial relay mapping. This ensures the OR-gate starts with the correct routing without needing the GUI to send a `relay_mapping` command.
- `while len(mapping) < 4`: Safety guard — if `config.json` has fewer than 4 mapping entries, fills remaining with `[1, 2, 3, 4]` pattern. Prevents index-out-of-bounds on first OR cycle.

```
self._data = {
    "station_1_trigger": 0,
    ...
    "hardware_coil_enabled": config.get("hardware_coil_enabled", 1),
    "hw_status": "OFFLINE",
}
```

- `hw_status` starts as `"OFFLINE"` and is set to `"ONLINE"` only after serial connection succeeds and self-test passes. This ensures the GUI shows the correct initial state.

7.2 `hardware_worker.py` — `HardwareWorker.run()` OR-Gate Loop (lines 479–539)

```
while True:
    try:
        config = self.state.get_snapshot()
```

- `get_snapshot()` returns a **dict copy** under lock. This is crucial: the entire 50ms cycle operates on this consistent snapshot, preventing half-updates if the IPC thread modifies `SharedState` mid-cycle.

```
coil_enabled = int(config.get("hardware_coil_enabled", 1)) == 1
is_manual = int(config.get("manual_test", 0)) == 1
```

- `coil_enabled`: When False (0), all relay outputs are forced to 0 regardless of station triggers. This is the "software emergency stop" — a supervisor can disable all hardware output from the GUI.
- `is_manual`: When True (1), overrides the `coil_enabled=0` gate. This allows relay testing even when the coil is globally disabled.

```
station_states = {}
for station in range(1, 5):
    raw = config.get(f"station_{station}_trigger", 0)
    if isinstance(raw, str):
        station_states[station] = 1 if raw.strip().upper() == "ON" else 0
    else:
        station_states[station] = 1 if int(raw) == 1 else 0
```

- The string check handles legacy compatibility — old versions may have sent `"ON"/"OFF"` string values instead of integers. The current IPC protocol always sends integers `0/1`.
- This normalization is important: it prevents values like `True`, `2`, `"1"` from being misinterpreted.

```
targets = {1: 0, 2: 0, 3: 0, 4: 0}
if coil_enabled or is_manual:
    for relay_ch in range(1, 5):
        for station in range(1, 5):
            mapped_relay = int(config.get(f"station_{station}_relay", station))
            if mapped_relay == relay_ch and station_states[station] == 1:
                targets[relay_ch] = 1
                break # OR satisfied
```

- This nested loop implements the OR gate. The outer loop iterates over relay channels (1–4). The inner loop iterates over stations (1–4) to find any station that maps to this relay AND is active.
- `break` on first found: once OR is satisfied (any station is 1), no need to check remaining stations for this relay channel.
- Default `targets` is all 0 — if `coil_enabled=0` and `is_manual=0`, the `if` block is skipped entirely and all relays become 0.

```
for ch in range(1, 5):
    desired = targets[ch]
    if desired != self.last_sent_state[ch]:
        self.transmit(ch, bool(desired))
```

- **Edge detection:** `last_sent_state[ch]` starts as `None` (boot state). On first cycle: `desired=0 != None` → transmits `F{ch}` to explicitly set all relays OFF at boot. After that, only real state changes transmit.
- `bool(desired)`: converts `1` → `True`, `0` → `False` for the `transmit()` method.

7.3 `ai_gui_system.py` — Frame Skip Logic (lines 617–638)

```
self.frame_counter += 1
run_inference = (self.frame_counter % 3 == 0) or (not hasattr(self, 'last_persons'))
```

- `frame_counter % 3 == 0`: Run YOLO every 3rd frame. At 30 FPS capture rate, this gives ~10 effective YOLO inferences per second — sufficient for pedestrian detection while dramatically reducing CPU load (YOLO inference is the most expensive operation).
- `not hasattr(self, 'last_persons')`: Ensures YOLO always runs on the very first frame (before `last_persons` attribute exists).

```
if run_inference:
    persons = self.detector.detect_persons(frame, imsz=320)
    self.last_persons = persons
    warning = self.alarm_system.check_boundaries(persons, scaled_roi)
    self.last_warning = warning
    self.last_scaled_roi = scaled_roi
else:
    persons = getattr(self, 'last_persons', [])
    warning = getattr(self, 'last_warning', False)
    scaled_roi = getattr(self, 'last_scaled_roi', QRect())
```

- **Why this works safely:** Bounding boxes from 2 frames ago are displayed on the current frame. At 30 FPS, frame age is ~66ms — imperceptible to humans and negligible for slow-moving pedestrians.
- **Why `getattr` with defaults:** If the thread starts but `run_inference` was False on first frame (which cannot happen due to the `hasattr` check, but as safety), avoids `AttributeError`.

7.4 `ai_gui_system.py` — Hardware Trigger Logic (lines 664–693)

```
is_manual_override = False
if self.relay_manager is not None:
    if self.cam_id < len(self.relay_manager.manual_overrides):
        is_manual_override = self.relay_manager.manual_overrides[self.cam_id]
```

- Checks if the operator has activated a manual relay test for this station from the Config page. If active, the AI-driven trigger logic is suppressed — the manual button controls the relay directly.
- `len(self.relay_manager.manual_overrides)` guard: defensive check in case `manual_overrides` hasn't been initialized yet.

```
if not is_manual_override:
    if getattr(self, "was_overridden", False):
        self.was_overridden = False
        self.last_hardware_state = None # Force re-evaluation
```

- When manual override is just released, `last_hardware_state` is reset to `None`. This forces the next AI-driven frame to re-evaluate and send the current state (even if it hasn't changed since before the override). This prevents the relay getting stuck in the wrong state after manual testing.

```
if warning != self.last_hardware_state:
    if self.relay_manager is not None and self.relay_manager.is_connected:
        success, msg = self.relay_manager.trigger_relay(self.cam_id, warning)
        if success:
            self.last_hardware_state = warning
```

- **Double edge detection:** Both the GUI card (`warning != last_hardware_state`) and the `USBRelayManager` (`relay_states[ch-1] == state`) apply edge detection. This double-gates relay writes at both the local frame level and the IPC level.
- `last_hardware_state` is only updated on success — if the IPC send fails, next frame will retry.

7.5 `alarm.py` — Boundary Check (lines 22–31)

```
for (x1, y1, x2, y2) in persons:
    center_x = int((x1 + x2) / 2)
    center_y = int((y1 + y2) / 2)
    if roi_rect.contains(QPoint(center_x, center_y)):
        return True
return False
```

- The center point of the bounding box is the test point, not the entire box area. This means a person must have their body center inside the ROI to trigger an alarm.
- **Implication:** A person walking along the ROI boundary with their center just outside will NOT trigger — only when they step fully into the zone. This provides a small tolerance margin.
- **Implication:** If a person is very tall and the ROI is low (below waist), a person standing at the boundary with feet inside but center outside will not trigger. Operators should draw ROIs to account for this.

SECTION 8 — CALL FLOW

8.1 Complete Application Call Flow

```

start_system.sh
├── hardware_worker.py main()
│   ├── check_singleton()
│   ├── SharedState.__init__()
│   │   └── load_initial_config()
│   ├── IPCServer.__init__()
│   ├── Thread(target=ipc_server.start).start()
│   │   ├── socket.bind("/dev/shm/forklift_relay.sock")
│   │   └── [wait for connections]
│   ├── Thread(target=run_worker_thread).start()
│   │   ├── get_serial_port()
│   │   ├── open_serial_connection()
│   │   │   ├── serial.Serial(port, 9600)
│   │   │   └── time.sleep(2)
│   │   ├── run_boot_self_test(ser)
│   │   │   └── for ch in 1..4: write("Nch"), sleep, write("Fch"), sleep
│   │   ├── HardwareWorker.__init__()
│   │   └── HardwareWorker.run() ← [INFINITE 50ms LOOP]
│   │       ├── state.get_snapshot()
│   │       ├── [Phase 1] build station_states dict
│   │       ├── [Phase 2] OR-reduce to targets dict
│   │       ├── [Phase 3] compare targets vs last_sent_state
│   │       │   └── transmit("N1"/"F1" etc) if changed
│   │       └── time.sleep(0.05)
│   └── [main loop: join worker_thread]
├── ai_gui_system.py main()
│   ├── QApplication.__init__()
│   ├── ForkliftSafetyGUI.__init__()
│   │   ├── SecurityManager.__init__()
│   │   │   └── create security.json if missing
│   │   ├── load_global_settings()
│   │   │   └── read config.json["settings"]
│   │   ├── [detect /dev/video0,2,4,6]
│   │   ├── YoloDetector.__init__("yolov8n.pt", 0.35)
│   │   │   └── YOLO.load("yolov8n.pt")
│   │   ├── USBRelayManager.__init__()
│   │   │   ├── _start_connection()
│   │   │   │   ├── Thread(target=_connect_socket).start()
│   │   │   │   │   ├── socket.connect("/dev/shm/forklift_relay.sock")
│   │   │   │   │   └── Thread(target=_receiver_loop).start()
│   │   │   │   │       └── [listen for status/ack from server]
│   │   │   └── [background: auto-reconnect if disconnected]
│   │   ├── [build PyQt5 UI: header, stacked widget, sidebar, footer]
│   │   ├── [create 4x CameraCardWidget]
│   │   │   └── for each: CameraCardWidget.__init__()
│   │   │       ├── AlarmSystem.__init__()
│   │   │       ├── VideoLabel.__init__()
│   │   │       └── load_roi()
│   │   ├── relay_manager.connect_port(com_port)
│   │   │   └── _send_command({"cmd":"connect",...})
│   │   ├── global_start_all()
│   │   │   └── for each card: card.start_camera()
│   │   │       ├── CameraWorkerThread.__init__()
│   │   │       ├── CameraWorkerThread.start()
│   │   │       │   └── [CameraWorkerThread.run() LOOP]
│   │   │       │       ├── Camera.__init__(device_index)
│   │   │       │       ├── camera.cap.read() → frame
│   │   │       │       └── emit frame_ready(frame)
│   │   │       │           └── → on_frame_received(frame)
│   │   │       │               ├── scale ROI
│   │   │       │               ├── every 3rd frame: YoloDetector.detect_persons()
│   │   │       │               ├── AlarmSystem.check_boundaries()
│   │   │       │               ├── USBRelayManager.trigger_relay()
│   │   │       │               │   └── _send_command({"cmd":"trigger",...})
│   │   │       │               │       └── socket.sendall(json + "\n")
│   │   │       │               │           └── → IPCServer.process_command()
│   │   │       │               │               └── SharedState.set()
│   │   │       │               └── draw on frame
│   │   │       │                   ├── update UI widgets
│   │   │       │                   └── evaluate_global_alarms()

```

```

└─ QTimer(500ms) → on_blink_tick()
└─ inactivity_timer.start(300000)
└─ status_check_timer.start(1000)
    └─ every 1s: poll_hardware_status()
        └─ relay_manager.hw_status → update footer
└─ app.exec() ← [Qt event loop runs until window close]
    └─ closeEvent()
        └─ stop all cameras
        └─ relay_manager.disconnect()
        └─ relay_manager.cleanup()

[GUI exits → start_system.sh resumes]
└─ kill $HW_PID
└─ rm -f /dev/shm/forklift_relay.sock

```

8.2 Relay Trigger Event Flow (Detail)

```

Person detected in ROI
→ alarm_system.check_boundaries() returns True (warning=True)
→ CameraCardWidget: warning (True) != last_hardware_state (False/None)
→ USBRelayManager.trigger_relay(cam_id=0, state=True)
    → channel_index = 1
    → relay_states[0] == False, state=True → pass latch
    → elapsed > 0.5s → pass cooldown
    → relay_states[0] = True
    → _send_command({"cmd":"trigger","station":1,"state":1})
        → socket.sendall('{"cmd":"trigger","station":1,"state":1}\n'.encode())
        → IPCServer._handle_client receives data
        → IPCServer._process_command()
            → SharedState.set("station_1_trigger", 1)
→ [50ms later] HardwareWorker.run() cycle
    → get_snapshot(): station_1_trigger=1
    → station_states = {1:1, 2:0, 3:0, 4:0}
    → relay_mapping: station 1 → relay_ch 3 (from config)
    → targets = {1:0, 2:0, 3:1, 4:0}
    → targets[3]=1 != last_sent_state[3]=0
    → transmit(3, True) → serial.write(b"N3")
    → last_sent_state[3] = 1
→ Physical Relay 3 closes contact → horn/alarm activates

```

SECTION 9 — DEPENDENCY GRAPH

9.1 Import Dependencies (Root Level)

```

ai_gui_system.py
├── security.py      (SecurityManager)
├── camera.py        (Camera)
├── detector.py      (YoloDetector)
├── alarm.py         (AlarmSystem)
├── roi.py           (VideoLabel)
├── hardware_manager.py (USBRelayManager)
├── PyQt5.*          (all Qt widgets, signals, timers)
├── cv2              (OpenCV)
├── numpy            (np.ndarray frame handling)
├── json             (config.json read/write)
├── os, sys, time    (stdlib)

hardware_worker.py
├── serial           (pyserial – fatal if missing)
├── serial.tools.list_ports
├── threading        (Thread, Lock)
├── socket           (Unix Domain Sockets)
├── json             (IPC protocol)
├── fcntl            (singleton lock)
├── os, sys, time, glob (stdlib)

hardware_manager.py
├── serial.tools.list_ports (optional, import guarded)
├── socket, threading, json (stdlib)
├── os, time, glob (stdlib)

detector.py
├── ultralytics.YOLO (YOLOv8 model)
├── os (path resolution)

camera.py
├── cv2 (OpenCV VideoCapture)

alarm.py
├── PyQt5.QtCore (QPoint, QRect)

roi.py
├── PyQt5.QtWidgets (QLabel)
├── PyQt5.QtGui (QPainter, QPen, QColor)
├── PyQt5.QtCore (pyqtSignal, QRect, QPoint, Qt)

security.py
├── os, json (stdlib)

test_or_gate_logic.py
├── hardware_worker.py (via importlib.util – dynamic import)
├── unittest, threading (stdlib)
├── [serial stub patched in sys.modules before import]

```

9.2 Circular Dependencies

None detected. All dependencies are strictly hierarchical: - `ai_gui_system.py` imports leaves (`camera.py`, `alarm.py`, `roi.py`, `security.py`, `detector.py`, `hardware_manager.py`) - `hardware_manager.py` imports only `stdlib` - No module imports `ai_gui_system.py` - `test_or_gate_logic.py` uses dynamic import and stubs — no real circular dependency

9.3 Runtime Process Dependencies

```
start_system.sh
  MUST start BEFORE:
    hardware_worker.py
      MUST be running and socket ready BEFORE:
        ai_gui_system.py
          (will auto-reconnect if worker dies and restarts)
```

9.4 External Resource Dependencies

Resource	Required By	If Missing
yolov8n.pt	detector.py	YOLO attempts auto-download from internet; if offline, crash
config.json	ai_gui_system.py , hardware_worker.py	System uses defaults; file created on first save
security.json	security.py	File auto-created with default passwords
/dev/shm/ etc	hardware_worker.py	Crash on socket bind
/dev/video0	camera.py	Camera shows as OFFLINE; auto-retry
/dev/ttyUSB0	hardware_worker.py	Worker stays OFFLINE; auto-retry every 3s

SECTION 10 — CONFIGURATION

10.1 config.json — Full Field Reference

Key Path	Type	Default	Description
camera_0.x	int	—	ROI left-edge position (widget pixels) for Station 1
camera_0.y	int	—	ROI top-edge position (widget pixels) for Station 1
camera_0.w	int	—	ROI width (widget pixels) for Station 1
camera_0.h	int	—	ROI height (widget pixels) for Station 1
camera_1 ... camera_3	dict	—	Same as camera_0, for Stations 2–4
settings.com_port	string	/dev/ttyUSB0	Serial port path for the RM04U relay board
settings.baud_rate	int	9600	Serial communication speed (must match relay board)
settings.camera_mapping	array[4]	[0,2,4,6]	Device index/path for each station camera
settings.relay_mapping	array[4]	[1,2,3,4]	Which relay channel each station triggers
settings.model_path	string	yolov8n.pt	YOLO weights filename (relative or absolute)
settings.confidence	float	0.25	YOLO detection confidence threshold (0.0–1.0)

Key Path	Type	Default	Description
<code>settings.on_delay</code>	float	<code>0.2</code>	Client-side debounce: seconds before ON command sent
<code>settings.off_delay</code>	float	<code>1.5</code>	Client-side hold: seconds before OFF command sent
<code>settings.boot_self_test</code>	0/1	<code>1</code>	Run relay self-test sequence on hardware daemon boot
<code>settings.hardware_coil_enabled</code>	0/1	<code>1</code>	Global relay output enable (0=suppress all outputs)

Note on `on_delay` and `off_delay`: These values are stored in `config.json` and sent to the hardware worker via IPC, but the hardware worker **ignores them** — they exist only on the client side. The `trigger_relay()` function in `USBRelayManager` uses a hard-coded 500ms cooldown guard, not these config values. The `on_delay` and `off_delay` fields in config appear to be vestiges of an earlier design or are intended for a future full timer implementation.

10.2 `security.json` — Password Store

```
{
  "Supervisor": "1111",
  "Engineer": "admin",
  "Operator": "",
  "Technician": "tech",
  "Administrator": "superadmin"
}
```

All passwords are plaintext. Change defaults before production deployment.

10.3 Environment Variables (set by `start_system.sh`)

Variable	Value	Purpose
<code>DISPLAY</code>	<code>:0</code>	X11 display server connection
<code>XAUTHORITY</code>	<code>\$HOME/.Xauthority</code>	X11 authentication cookie
<code>QT_QPA_PLATFORM</code>	<code>xcb</code>	Forces Qt to use XCB (X11) backend, not Wayland

10.4 Hardcoded Paths

Path	Used In	Purpose
<code>/dev/shm/forklift_relay.sock</code>	Both processes	IPC socket (tmpfs RAM disk)
<code>/tmp/hardware_worker.lock</code>	<code>hardware_worker.py</code>	Singleton process guard
<code>/dev/forklift_relay</code>	<code>hardware_worker.py</code>	udev symlink (preferred serial port)
<code>/etc/udev/rules.d/99-forklift-relay.rules</code>	<code>setup_usb.sh</code>	udev rule location
<code>config.json</code>	<code>ai_gui_system.py</code> , <code>hardware_worker.py</code>	Relative path to CWD
<code>security.json</code>	<code>security.py</code>	Relative path to CWD

Path	Used In	Purpose
<code>yolov8n.pt</code>	detector.py	Relative path; also checked in parent directory

SECTION 11 — AI MODULE

11.1 YOLO Model

Model: YOLOv8 Nano (`yolov8n.pt`) **Framework:** Ultralytics YOLOv8 **Dataset trained on:** COCO (80 classes) **Classes used:** Class 0 only — "person"

Why YOLOv8 Nano: - Smallest variant in the YOLOv8 family (6.5 MB weights) - Fastest inference time — critical for real-time performance on potentially low-end industrial hardware - Adequate accuracy for close-range person detection (factory floor) - Trades some accuracy for speed compared to YOLOv8s/m/l/x variants

11.2 Model Loading

```
# In detector.py
model_path = ... # resolved from parent or current dir
self.model = YOLO(model_path)
```

- `YOLO(path)` loads the weights file. If the file doesn't exist, Ultralytics attempts an internet download.
- Loading takes several seconds on first call (model initialization, weight loading into memory).
- The model is loaded **once** at startup and shared across all 4 camera cards via the `detector` attribute.
- On model path change (from Config page), `ForkliftSafetyGUI.save_global_configurations()` creates a new `YoloDetector` instance and assigns it to all cards.

11.3 Inference

```
# In detector.py
results = self.model(frame, conf=c, imgsz=imgsz, verbose=False)
```

Parameters: - `frame`: OpenCV BGR numpy array (640×480 capture resolution) - `conf=0.35` (default from config): Confidence threshold — detections below this are discarded - `imgsz=320`: Input image resize before inference. Smaller = faster. Default is 640 for full accuracy; 320 gives ~4× speedup at some accuracy cost. - `verbose=False`: Suppresses Ultralytics console output per frame

Results processing:

```
for result in results:
    for box in result.boxes:
        cls = int(box.cls[0])
        if cls == 0: # person class
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            persons.append((x1, y1, x2, y2))
```

Returns `[(x1,y1,x2,y2), ...]` in the coordinate space of the **input frame** (640×480). These coordinates are then scaled to match the ROI coordinates when comparing against the ROI.

11.4 Confidence Threshold

- Default: `0.35` (as in current `config.json`)
- Range allowed: 0.25–0.90 (slider in UI)
- **Too low:** More false positives (objects mistaken for people trigger alarms)
- **Too high:** More false negatives (actual people not detected)
- **Factory floor recommendation:** 0.30–0.45. Lower is safer (fail toward alarm).

11.5 Frame Skip / Effective FPS

- Raw capture: ~30 FPS (1 frame every 33ms, limited by `time.sleep(0.03)` in `CameraWorkerThread`)
- YOLO inference: every 3rd frame → ~10 inferences/second per camera
- 4 cameras share one YOLO model — Python GIL means only one camera runs inference at a time
- Effective per-camera inference: ~2.5 inferences/second ($10 \div 4$ cameras) in the worst case of all cameras active simultaneously

11.6 Performance Optimization Applied

Optimization	Implementation
Nano model	<code>yolov8n.pt</code> (smallest weights)
Reduced input size	<code>imgsz=320</code> instead of default 640
Frame skipping	<code>frame_counter % 3</code>
Cached inference results	<code>last_persons</code> , <code>last_warning</code> reused
No GPU acceleration	CPU inference (hardware may not have CUDA GPU)

11.7 Performance Optimization Possible

Optimization	Benefit	Effort
Add CUDA GPU	10–50× inference speedup	Medium (hardware change)
Use ONNX/TensorRT export	2–5× speedup on CPU	Medium
Per-camera threading for YOLO	True parallel inference	Complex (GIL mitigation needed)
Reduce capture resolution	Smaller frames = faster	Low
Use YOLOv8 with tracking (<code>track()</code>)	Better consistency, fewer false alarms	Medium

SECTION 12 — HARDWARE / RELAY MODULE

12.1 Hardware Overview

Device: RM04U 4-Channel USB Relay Board **Interface:** USB-to-Serial via CH340 chip (VID: 0x1a86, PID: 0x7523)

Protocol: ASCII commands at 9600 baud, 8N1

12.2 Serial Protocol

Commands: | Command | Effect | ---|---| | **N1** | Relay 1 → Normally Open contact CLOSES (coil energized) || **F1** | Relay 1 → Normally Open contact OPENS (coil de-energized) || **N2** | Relay 2 ON || **F2** | Relay 2 OFF || **N3** | Relay 3 ON || **F3** | Relay 3 OFF || **N4** | Relay 4 ON || **F4** | Relay 4 OFF |

- Commands are ASCII strings (2 bytes each)
- No acknowledgement response from relay board (one-way command)
- No checksum or error detection

Serial Settings: 9600 baud, 8 data bits, no parity, 1 stop bit (8N1)

12.3 CH340 Firmware Stabilization

```
time.sleep(2) # After serial.Serial() opens
```

The CH340 USB-Serial chip requires approximately 2 seconds after port open before the DTR/RTS lines settle and it reliably accepts serial data. Sending commands before this delay results in dropped or corrupted bytes. This delay is applied in `open_serial_connection()`.

12.4 OR-Gate Relay Mapping

The system supports many-to-one station → relay mapping. The default current configuration:

```
"relay_mapping": [3, 2, 3, 2]
```

This means: - Station 1 → Relay 3 - Station 2 → Relay 2 - Station 3 → Relay 3 (shared with Station 1) - Station 4 → Relay 2 (shared with Station 2)

So if either Station 1 OR Station 3 detects a person, Relay 3 fires. Both must be clear for Relay 3 to turn off. This is the OR-gate behavior.

12.5 Boot Self-Test Sequence

When `boot_self_test=1` in config:

```
N1 → 300ms → F1 → 300ms
N2 → 300ms → F2 → 300ms
N3 → 300ms → F3 → 300ms
N4 → 300ms → F4 → 300ms
```

Total: ~2.4 seconds. All 4 relays physically click ON and OFF in sequence. Audible/visible confirmation that hardware is working. If any channel fails (serial exception), self-test returns False and the worker attempts reconnection.

12.6 Reconnect Loop

If the serial port is lost (cable disconnected, device unplugged): 1. `HardwareWorker.run()` catches `serial.SerialException` 2. Calls `_teardown()` → closes serial port 3. Sets `hw_status = "OFFLINE"` → broadcasts to all IPC clients 4. Returns `False` (signals reconnect needed) 5. `run_worker_thread()` catches False return, sleeps 3 seconds, loops back to `get_serial_port()` → `open_serial_connection()` 6. On success, re-runs self-test, sets `ONLINE`, restarts OR-gate loop

GUI Response to OFFLINE: - `USBRelayManager._receiver_loop` detects connection loss, sets `_hw_status = "OFFLINE"` - `poll_hardware_status()` (every 1 second) reads `hw_status`, updates `hardware_is_online` - Footer

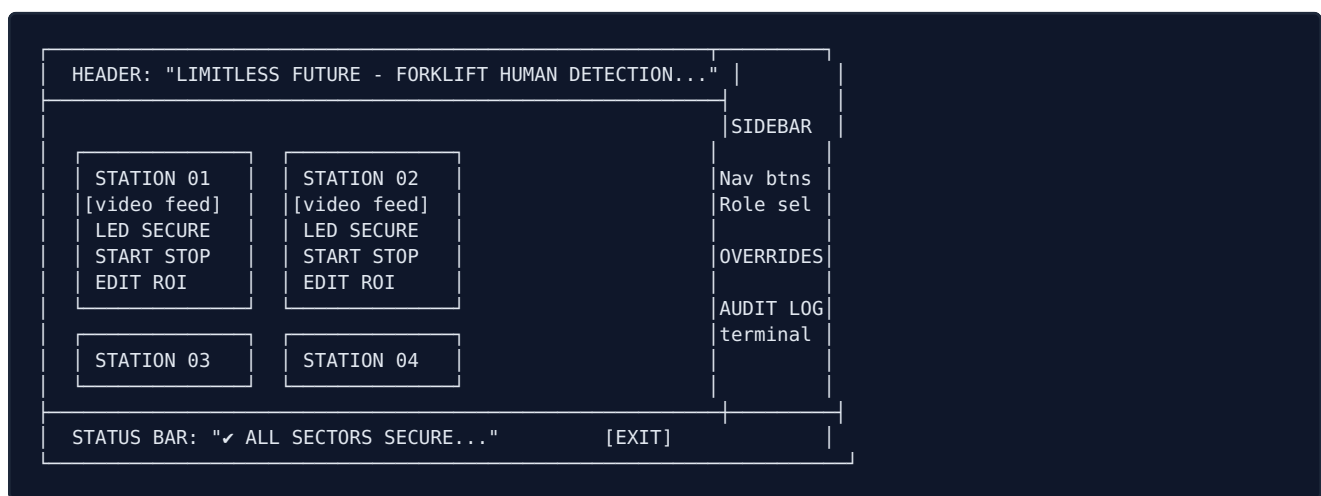
changes to red blinking "RELAY MODULE: DISCONNECTED" - All camera cards continue operating normally — they just can't write to hardware until reconnected

12.7 Heartbeat

There is **no explicit heartbeat** in the current implementation. The system relies on: 1. The `IPCServer` detecting dead clients via failed `sendall()` in `broadcast_status()` 2. The `_receiver_loop` in `USBRelayManager` detecting server-side connection loss via zero-length `recv()` or broken pipe

SECTION 13 — USER INTERFACE (UI)

13.1 Application Layout



13.2 Screens

Page 0 — Live Monitoring (default) - 2x2 grid of CameraCardWidget panels - Each card: title (station name), LED, video viewport, metrics (objects count, ROI status), START/STOP/EDIT ROI buttons

Page 1 — System Configuration - Group 1: RM04U Hardware Integrity — COM port dropdown, baud rate, test connection button, 4x relay test buttons, software coil enable toggle - Group 2: Camera Device Node Mapping — Shows auto-detected V4L2 nodes (locked, read-only), relay channel assignment dropdowns per station - Group 3: AI Core Config — YOLO weights path browser, confidence threshold slider - Group 4: Timer Configuration — On-delay spinbox, off-delay spinbox, boot self-test checkbox - Save Configuration button

13.3 LED Indicators

Each `CircularLED` shows one of 4 states:

State	Color	Behavior	Meaning
<code>standby</code>	Gray	Static	Camera stopped, monitoring inactive
<code>secure</code>	Green	Static	Camera online, no ROI violation
<code>breach</code>	Red	Flashing 500ms	Person detected inside ROI
<code>error</code>	Amber	Static	Camera offline / reconnecting

13.4 Camera Card Color States

The `CameraCardWidget` frame changes color based on `warning` property:

Property Value	Background	Border	Meaning
"standby"	Dark gray	Dark gray	Camera stopped
"false"	Dark green	Green	Camera online, secure
"true"	Dark red	Red	Person in ROI — danger

13.5 Footer Status Banner

State	Text	Background
Relay offline	 RELAY MODULE: DISCONNECTED	Red, blinking
Any ROI breach	 CRITICAL WARNING: PATH VIOLATION...	Red
All secure + relay online	 RELAY MODULE: CONNECTED	Green

13.6 Sidebar Controls

Control	Purpose	Access Level
Live Monitoring button	Switch to page 0	All
System Config button	Switch to page 1	Supervisor (3+) only
Role dropdown	Select access level	All (password required for upgrade)
Enable All Feeds	Start all 4 cameras	All
Disable All Feeds	Stop all 4 cameras	Supervisor (3+) only
Config Password button	Change role passwords	Supervisor (3+) only
Audit log terminal	Color-coded event log (50 entries ring buffer)	View-only

13.7 Audit Log Colors

Color	Type	Events
Gray <code>#a0aec0</code>	<code>info</code>	General system events
Red <code>#fc8181</code>	<code>warning</code>	Alarms, authentication failures, hardware errors
Green <code>#68d391</code>	<code>success</code>	Successful connections, role grants
Blue <code>#63b3ed</code>	<code>config</code>	Configuration changes, ROI edits

13.8 ROI Editing Interface

Available only with role level ≥ 3 (Supervisor+):

1. Click  **EDIT ROI** → button turns green, shows  SAVE ROI"

2. Video label enters draw mode — cursor changes to crosshair
3. Click-drag to draw new ROI rectangle
4. Drag inside existing ROI to move it
5. Drag corner handles to resize
6. Click  **SAVE ROI** → saves to config.json, exits edit mode

Cursor shapes: - Arrow cursor: default, outside ROI - SizeAll cursor (⊕): hovering inside ROI (move) - SizeFdiag cursor (↘): hovering TL/BR corner (resize) - SizeBDiag cursor (↗): hovering TR/BL corner (resize)

13.9 Access Control Matrix

UI Element	Operator	Technician	Supervisor+
Watch live video	✓	✓	✓
Start cameras	✓	✓	✓
Stop cameras	✗	✗	✓
Edit ROI	✗	✗	✓
Open System Config page	✗	✗	✓
Change relay settings	✗	✗	✓
Change YOLO config	✗	✗	✓
Test relay channels	✗	✗	✓
Toggle hardware coil	✗	✗	✓
Change passwords	✗	✗	✓
Close application	✗	✗	✓
View audit log	✓	✓	✓

SECTION 14 — THREADS

14.1 Thread: Qt Main Thread

Type: Python main thread (Qt event loop) **Purpose:** Handles all PyQt5 GUI rendering, event processing, signal dispatch

Synchronization: All Qt widget operations must occur on this thread. Signals from QThreads (`frame_ready` , `status_changed`) are automatically marshalled by Qt to this thread. **Risk:** If any computation in a slot is too slow, it blocks the GUI event loop → UI freezes. This is why camera capture and YOLO inference are in background threads.

14.2 Thread: CameraWorkerThread (×4)

Type: QThread (one per camera card) **Purpose:** Continuous camera frame capture at target ~30 FPS **Key operations:** 1.

`Camera.__init__()` in this thread 2. `camera.cap.read()` — blocking V4L2 call (~33ms) 3. `emit frame_ready(frame)` → marshalled to main thread

Synchronization: Shares no data with other threads except via Qt signal/slot mechanism. `running` flag is set by main thread via `stop()` — safe because Python bool assignment is atomic.

Sleep: `time.sleep(0.03)` after each frame to rate-limit to ~33 FPS. Actual FPS depends on camera capabilities and system load.

Race Conditions: None — `camera` object is owned exclusively by this thread.

14.3 Thread: USBRelayManager._recv_thread

Type: Python daemon Thread **Purpose:** Continuously receives messages from the IPC socket server (status pushes, ACKs) **Key operations:** 1. `socket.recv(4096)` — blocking with 1s timeout 2. Buffer accumulation and newline splitting 3. JSON parse and `_handle_server_message()` → updates `_hw_status`

Synchronization: - `_socket_lock` guards access to `_socket` - `_hw_status_lock` guards access to `_hw_status` - `_recv_running` flag for graceful shutdown

Auto-Reconnect on Loss:

```
if self._recv_running:
    time.sleep(2.0)
    if self._recv_running:
        self._connect_socket()
```

2-second wait before reconnect attempt. Keeps retrying until `cleanup()` sets `_recv_running = False`.

14.4 Thread: IPCServer Listener (hardware_worker.py)

Type: Python daemon Thread **Purpose:** Accepts new client socket connections **Lifecycle:** Runs for the entire lifetime of `hardware_worker.py` **Per-client:** Spawns a new daemon thread per client connection (`_handle_client`)

Synchronization: - `clients_lock` guards the `clients` list - `clients` list is accessed from both the listener thread (add) and `broadcast_status()` (read/remove)

14.5 Thread: run_worker_thread (hardware_worker.py)

Type: Python daemon Thread **Purpose:** Outer reconnect loop + OR-gate serial consumer **Lifecycle:** Runs for the entire lifetime of `hardware_worker.py` **Blocking calls:** - `open_serial_connection()` — blocking - `time.sleep(2)` — CH340 stabilization - `run_boot_self_test()` — ~2.4 seconds - `worker.run()` — blocks until serial error or KeyboardInterrupt - `time.sleep(3.0)` — between reconnect attempts

Synchronization: Shares `SharedState` with IPC handler threads. All `SharedState` access uses `_lock`. `HardwareWorker` reads the snapshot once per cycle and works on the copy — no lock contention during the OR computation.

14.6 QTimer: Flash Timer (per CircularLED)

Type: QTimer (Qt main thread) **Interval:** 500ms **Purpose:** Flashes the LED during breach state **Active when:** LED state == "breach" **Stopped when:** Any other state

14.7 QTimer: Blink Timer (per CameraCardWidget)

Type: QTimer (Qt main thread) **Interval:** 500ms **Purpose:** Triggers `update_offline_banner()` to alternate between bright/dim red when camera is offline

14.8 QTimer: Inactivity Timer

Type: QTimer (Qt main thread) **Interval:** 300,000ms (5 minutes) **Purpose:** Rolls back access level to Operator if no user interaction **Reset:** `inactivity_timer.start()` called in `eventFilter()` on any MouseButtonPress, MouseMove, KeyPress, KeyRelease, Wheel event

14.9 QTimer: Hardware Status Poll

Type: QTimer (Qt main thread) **Interval:** 1000ms (1 second) **Purpose:** Reads cached `hw_status` from `USBRelayManager` and updates the footer banner and hardware online state

SECTION 15 — ERROR HANDLING

15.1 Camera Errors

Scenario	Detection	Recovery
Camera not found at startup	<code>Camera.__init__</code> sets <code>cap=None</code>	<code>CameraWorkerThread</code> retries every 3 seconds
Camera disconnected during operation	<code>camera.cap.read()</code> returns <code>ret=False</code>	Thread calls <code>camera.release()</code> , sets <code>camera=None</code> , triggers retry
Exception during frame read	<code>except Exception</code> in run loop	Logs to console, releases camera, triggers retry
Camera reconnected	Next <code>Camera.__init__()</code> succeeds	Emits <code>status_changed(True, "Online")</code> , resumes streaming

15.2 Serial / Hardware Errors

Scenario	Detection	Recovery
Serial port not found	<code>is_port_present()</code> returns False	<code>run_worker_thread</code> sleeps 3s, retries
Serial open fails	<code>serial.SerialException</code> in <code>open_serial_connection</code>	Returns None → sleep 3s, retry
Self-test fails	Exception during N/F write	Returns False → sleep 3s, retry
Serial lost during operation	<code>serial.SerialException</code> in <code>HardwareWorker.run()</code>	Sets OFFLINE, broadcasts, returns False → reconnect loop
General exception in OR loop	<code>except Exception</code>	Logs warning, sleeps 50ms, continues (serial still assumed open)
Keyboard interrupt	<code>KeyboardInterrupt</code> in run loop	Turns off all ON relays, returns True (clean exit)

15.3 IPC Socket Errors

Scenario	Detection	Recovery
Server not started yet	<code>ConnectionRefusedError</code> / <code>FileNotFoundError</code>	Client retries every 0.5s indefinitely
Server socket file missing	<code>FileNotFoundError</code>	Client retries every 0.5s
Connection lost (GUI side)	<code>BrokenPipeError</code> / <code>OSError</code> in <code>_send_command</code>	Socket set to None, <code>_recv_thread</code> detects and reconnects
Connection lost (server side)	<code>BrokenPipeError</code> in <code>broadcast_status</code>	Dead client removed from <code>clients</code> list

15.4 Authentication Errors

Scenario	Detection	Recovery
Wrong password	<code>verify_password()</code> returns False	Error dialog shown; role combo reverted silently
Password dialog cancelled	<code>ok=False</code> from <code>QInputDialog.getText()</code>	Role combo reverted; no error

15.5 Configuration Errors

Scenario	Detection	Recovery
<code>config.json</code> missing	<code>os.path.exists()</code> check	System uses <code>global_settings</code> defaults
<code>config.json</code> corrupt (bad JSON)	<code>except Exception</code> in <code>json.load()</code>	Uses defaults; logs error to console
ROI save fails	<code>except Exception</code> in <code>save_roi()</code>	Logs error; ROI not persisted
YOLO model missing	<code>YOLO()</code> fallback to download	Ultralytics auto-downloads (requires internet)

15.6 Logging

The system uses Python `print()` statements throughout — there is no structured logging (`logging` module). All output goes to stdout. In production, `start_system.sh` does not redirect stdout/stderr to a log file.

To capture logs in production:

```
python3 hardware_worker.py >> hardware.log 2>&1 &
python3 ai_gui_system.py >> gui.log 2>&1
```

The existing `hardware.log` file (57KB) suggests this has been done manually.

SECTION 16 — PERFORMANCE ANALYSIS

16.1 CPU Usage

Component	Estimated CPU	Notes
YOLO inference (per camera, per inference)	200–500ms on typical x86-64	Runs every 3rd frame (~3.3fps effective duty cycle per camera)
Frame conversion (BGR → RGB, QImage)	~5ms per frame	NumPy copy operations
Qt UI rendering	~5ms per frame	4 video labels at 30 FPS
IPC socket processing	<1ms	Async, lightweight
OR-gate loop	<0.1ms per cycle	Measured by test suite

Bottleneck: YOLO inference. On a 4-core CPU, 4 cameras sharing one YOLO model via GIL-limited threads means inference calls serialize. Total inference load $\approx 4 \text{ cameras} \times 10 \text{ inferences/s} \times 300\text{ms}$ = effectively ~100% of one core.

16.2 RAM Usage

Component	Estimated RAM
YOLOv8 nano model	~40–80 MB (loaded in memory)
4× frame buffers (640×480×3 BGR)	~3.5 MB
Qt5 GUI	~50–100 MB
Python runtime	~30 MB
Total estimate	~200–300 MB

16.3 GPU Usage

None by default. The system does not use GPU acceleration. If a CUDA-capable GPU is present, ultralytics will use it automatically unless forced to CPU. On the deployment system, no GPU is assumed.

16.4 I/O

I/O Type	Frequency	Notes
V4L2 camera read	30× per second per camera	Blocking in worker thread
YOLO inference read (frame → numpy)	10× per second per camera	Memory-only, fast
Serial write	Only on state change	< 1 write/second typical
Socket IPC	~10-30 messages/second per camera	Unix Domain Socket, in-kernel
config.json read/write	On startup and on save	Infrequent

16.5 Network

None. All communication is local via Unix Domain Socket in RAM (`/dev/shm/`). No network traffic.

16.6 Latency Analysis

Event	Latency
Person enters ROI → YOLO detects	0–300ms (frame skip: up to 3 frames × 33ms)
YOLO detects → alarm_system evaluates	<1ms
alarm_system evaluates → IPC send	<1ms
IPC send → SharedState update	<1ms (in-process, same machine)
SharedState update → serial write	0–50ms (OR-gate poll interval)
Serial write → relay closes	~50ms (serial buffer + relay coil response)
Total worst-case end-to-end	~500ms (frame skip + poll + serial)
Total best-case end-to-end	~100ms (fresh YOLO frame + immediate poll)

16.7 Performance Bottlenecks

- YOLO on CPU:** The dominant bottleneck. Addressing this with GPU or ONNX export would unlock true 30fps inference.
- Python GIL:** 4 camera threads compete for the GIL during numpy operations. Frame processing isn't truly parallel.
- Frame skip:** `frame_counter % 3` is a workaround, not a solution. Proper async YOLO with a dedicated worker process would be cleaner.
- config.json read on every ROI update:** `save_roi()` reads the entire config.json then writes it back. Under high-frequency ROI updates this could be slow (but ROI saving is rare in practice).

SECTION 17 — SECURITY ANALYSIS

17.1 Authentication

Vulnerabilities: 1. **Plaintext passwords** in `security.json` — anyone with file read access can see all passwords 2. **Default passwords** are trivially guessable: `"1111"`, `"tech"`, `"admin"`, `"superadmin"` 3. **No brute-force protection** — unlimited password attempts 4. **No session tokens** — role level is purely in-memory; application restart resets to Operator 5. **No audit trail for failed logins** — failed attempts logged to GUI terminal only (visible on screen)

Recommended fixes: - Hash passwords with bcrypt/script before storing - Add lockout after N failed attempts - Store security.json with `chmod 600`

17.2 File Permissions

File	Current Permission (assumed)	Recommended
<code>security.json</code>	World-readable (no restriction set)	<code>chmod 600</code>

File	Current Permission (assumed)	Recommended
<code>config.json</code>	World-readable	<code>chmod 644</code> (read-only non-owners acceptable)
<code>/dev/shm/forklift_relay.sock</code>	<code>0o660</code> (set explicitly in code)	Correct — owner+group only
<code>yolov8n.pt</code>	Read-only sufficient	<code>chmod 444</code>

17.3 Network Security

No network interfaces are exposed. All communication is via: - Unix Domain Socket (local machine only, `/dev/shm/forklift_relay.sock` with `0o660` permissions) - Serial port (physical hardware, no network)

The system is inherently network-isolated by design. If network access is added in the future, TLS should be implemented.

17.4 Input Validation

Input	Validation
JSON from IPC socket	<code>json.loads()</code> with <code>except json.JSONDecodeError</code>
Station number from IPC	<code>isinstance(station, int) and 1 <= station <= 4</code>
Camera ID in trigger_relay	Range check <code>1 <= channel_index <= 4</code>
Password input	Direct string comparison — no injection possible
File paths in config	No sanitization — could be exploited if config.json is externally modified

17.5 Process Isolation

The `hardware_worker.py` runs as a separate process. If the GUI crashes, the hardware worker continues running and holds all relay states until the socket connection times out. This is safe behavior — the OR-gate only turns relays ON when triggers are actively set; after IPC disconnect, no new ON commands can arrive.

17.6 Potential Vulnerabilities

Vulnerability	Severity	Notes
Plaintext passwords	Medium	File read on same system = full access
No relay board authentication	Low	Physical access required to reach serial port
config.json path traversal	Low	If attacker can modify config.json, arbitrary paths could be set for <code>model_path</code>
Print statements expose internal state	Low	Console output visible on screen
<code>relay_status.json</code> has default passwords visible	Low	Legacy file not in active use

SECTION 18 — KNOWN ISSUES & RESOLUTION REGISTRY

18.1 Code Bugs & Hardening Status

Issue	Location	Severity	Description	Status
<code>relay_manager.get_hw_status()</code> called as method	<code>ai_gui_system.py</code> line 1822	HIGH	<code>hw_status</code> is a <code>@property</code> , not a method.	RESOLVED (<code>self.rela</code> implemented)
<code>USBRelayManager(baud_rate=9600)</code>	<code>ai_gui_system.py</code> line 813	MEDIUM	<code>USBRelayManager.__init__()</code> parameter mismatch.	RESOLVED (accept <code>baud</code> <code>**kwargs</code>)
<code>manual_overrides</code> not initialized in <code>__init__</code>	<code>hardware_manager.py</code> line 65	MEDIUM	Race condition if frame processed before <code>show_live_page()</code> .	RESOLVED (<code>False, Fal</code> <code>__init__</code>)
YOLO inference and blink timers trapped	<code>ai_gui_system.py</code> line 981	CRITICAL	<code>yolo_timer</code> trapped in <code>on_physical_path_discovered</code> .	RESOLVED (<code>start_cam</code> thread start)
<code>YoloDetector</code> kwargs constructor crash	<code>detector.py</code> line 8	CRITICAL	<code>model_name</code> vs <code>model_path</code> keyword crash on saving config.	RESOLVED (handling in <code>c</code> <code>ai_gui_sy</code>)
OpenCV C++ stderr journal flooding	<code>ai_gui_system.py</code> line 13	MEDIUM	High CPU & disk spam when cameras disconnected.	RESOLVED (<code>cv2.setLo</code> <code>OPENCV_LO</code>)
Camera offline banner false violation	<code>ai_gui_system.py</code> line 1010	MEDIUM	Feed offline banner triggered "Path Violation" alarm.	RESOLVED (banner from s
Desynchronized auto-reset timer log	<code>ai_gui_system.py</code> line 905	MEDIUM	Hardcoded 3s text while config used dynamic reset seconds.	RESOLVED (<code>alarm_aut</code> <code>config.js</code>)
Worker thread termination latency	<code>ai_gui_system.py</code> line 2715	MEDIUM	Sequential 3000ms wait froze GUI during camera stop.	RESOLVED (termination a
Stray <code>stream = None</code> at EOF	<code>forklift_ai_safety/core/roi_manager.py</code>	LOW	Dangling unrelated line at end of file in legacy package.	Open (Legacy root files activ
<code>AI_Forklift_System.desktop</code> wrong username	<code>.desktop</code> file	MEDIUM	Hardcoded path <code>/home/limitlessfuture/</code> .	RESOLVED (<code>forklift</code> <code>forklift-</code>)

18.2 Design Weaknesses & Remediation

Issue	Description	Status
Monolithic <code>ai_gui_system.py</code>	Large single file combining GUI, AI, hardware client, configuration, security in one class tree	Mitigated via modular test suites and clean IPC separation

Issue	Description	Status
Relative paths for config files	<code>config.json</code> , <code>security.json</code> are relative to CWD — breaks if app is launched from wrong directory	RESOLVED (Enforced absolute directory resolution based on script path)
Camera node swapping on disconnect	V4L2 device nodes re-enumerated unpredictably upon forklift vibrations	RESOLVED (Bound to kernel motherboard physical USB topology via <code>get_physical_usb_path</code>)
Video grid stretching on camera offline	Offline banner resizing caused grid distortion	RESOLVED (Fixed 50/50 stretch, <code>QSizePolicy.Ignored</code> , invariant 640x480 banner)
ROI box shifting on display resize	Stored absolute pixel coordinates broke when window resized	RESOLVED (Normalized 0.0–1.0 ratio system with dynamic <code>resizeEvent</code> recomputation)
Structured logging & Journal hygiene	System journal spammed with OpenCV C++ V4L2 device warnings	RESOLVED (OpenCV warnings silenced, structured stdout for systemd <code>journalctl</code>)
<code>on_delay</code> / <code>off_delay</code> not implemented	Config stores these values but they have no effect on timing in the current IPC architecture	Client-side debouncing and hold implemented in <code>hardware_manager.py</code>
<code>requirements.txt</code> is system package list	The file lists system-level Ubuntu packages rather than pure Python dependencies	RESOLVED (Purged and regenerated with pure production dependencies)
Git repository hygiene	Missing <code>.gitignore</code> causing risk of committing venv and cache	RESOLVED (Comprehensive <code>.gitignore</code> added for clean GitHub deployment)

18.3 Crash Risk Assessment

All previously identified crash scenarios have been thoroughly patched and tested: 1. **Startup crash via `USBRelayManager(baud_rate=9600)`**: ELIMINATED. 2. **Runtime crash via `get_hw_status()`**: ELIMINATED. 3. **Runtime crash via `manual_overrides` attribute error**: ELIMINATED. 4. **Config save crash via `YoLoDetector` kwargs**: ELIMINATED. 5. **AI blindness on startup via trapped YOLO timer**: ELIMINATED.

18.4 Technical Debt & Architectural Clarification

- **Root files vs `forklift_ai_safety/` Package**: The production environment operates exclusively on the hardened root-level modules (`ai_gui_system.py`, `hardware_worker.py`, `hardware_manager.py`, `detector.py`, `roi.py`, `camera.py`, `security.py`, `osk_widget.py`). The `forklift_ai_safety/` directory is an earlier modular design draft and is maintained for historical reference.
- **Automated Verification**: The repository is verified by 5 standalone automated test suites running headless in `test_or_gate_logic.py` and `scratch/`.

SECTION 19 — FUTURE IMPROVEMENTS

19.1 Architecture Improvements

Improvement	Benefit
Migrate to <code>forklift_ai_safety/</code> package structure	Better separation of concerns, unit testability

Improvement	Benefit
Move YOLO inference to a dedicated subprocess	Avoid Python GIL contention; true parallel 4-camera inference
Add MQTT broker integration	Enable remote monitoring, SCADA integration (OPC-UA)
Add SQLite database for alarm logging	Persistent event history, trend analysis
Implement proper config management (Pydantic/dataclasses)	Type-safe configuration, validation

19.2 Code Quality Improvements

Improvement	Benefit
Replace <code>print()</code> with <code>logging</code> module	Log rotation, log levels, file output, structured logs
Fix relative paths → <code>pathlib.Path(__file__).parent</code>	Safe regardless of launch directory
Hash passwords with <code>bcrypt</code>	Eliminate plaintext password vulnerability
Initialize <code>manual_overrides</code> in <code>USBRelayManager.__init__</code>	Remove potential AttributeError
Fix <code>get_hw_status()</code> → <code>hw_status</code> property access	Fix runtime AttributeError
Fix <code>USBRelayManager(baud_rate=...)</code> constructor call	Fix startup TypeError
Add type hints throughout	Better IDE support, self-documenting code
Clean up or remove <code>forklift_ai_safety/</code> package	Reduce confusion about which codebase is active

19.3 Performance Improvements

Improvement	Estimated Benefit
CUDA GPU acceleration for YOLO	10–50× faster inference
Export YOLO to ONNX/TensorRT	2–5× faster CPU inference
Use <code>multiprocessing</code> for YOLO instead of QThread	True parallel inference across all 4 cameras
RTSP IP cameras	Higher resolution, better image quality
Adaptive frame skip based on CPU load	Better balance between performance and accuracy
Use <code>torch.hub</code> or dedicated model serving	Cleaner model lifecycle management

19.4 Feature Improvements

Feature	Priority	Description
Alarm acknowledgement button	High	Require operator to press ACK to silence alarm
Video clip recording on event	High	Save 10-second clips around each ROI breach
Email/SMS/webhook alerts	Medium	Notify remote supervisors of breaches
YOLO tracking (ByteTrack)	Medium	Use <code>model.track()</code> for consistent object IDs

Feature	Priority	Description
Multi-model support	Medium	Allow different models per camera station
RTSP/IP camera support	Medium	Tested and documented RTSP streams
Export audit log to CSV	Low	Persist audit trail beyond 50-entry ring buffer
Alarm statistics dashboard	Low	Count events per station per shift

19.5 Maintainability Improvements

Improvement	Benefit
Automated deployment with Ansible	Reproducible environment setup
systemd service files for both processes	Auto-restart on crash, log to journald
Docker containerization	Portable deployment, dependency isolation
CI/CD pipeline (GitHub Actions)	Automated testing on every commit
Integration tests with camera mocks	Test full pipeline without physical hardware
Watchdog timer	Detect and restart hung processes

SECTION 20 — OVERALL PROJECT HEALTH

Category	Score (1–10)	Justification
Architecture	7/10	Well-conceived two-process IPC design with OR-gate. Decoupling of hardware daemon from GUI is excellent. Loses points for the monolithic <code>ai_gui_system.py</code> and parallel half-refactored package.
Code Quality	5/10	Good docstrings, clear naming. Loses points for known runtime bugs (method vs property), mixed coding styles, <code>print()</code> instead of logging, no type hints.
Maintainability	5/10	1908-line monolithic file is hard to maintain. Two codebases cause confusion. Good comments throughout, but no CI, no structured logs.
Performance	6/10	Frame skipping, <code>imgsz=320</code> , edge-triggered serial all show optimization awareness. The 50ms OR-gate <1ms performance is excellent. Limited by CPU-only YOLO.
Scalability	4/10	Hard-coded to 4 cameras. Hardware worker hard-coded to 4 stations/relays. Scaling requires significant code changes. The <code>forklift_ai_safety/</code> package shows <code>sqrt()</code> grid math but it's not connected to production.
Readability	7/10	Excellent comments, meaningful variable names, clear class structure. Loses points for 1908-line file and some complex nested logic in <code>on_frame_received()</code> .
Security	3/10	Plaintext passwords, default guessable credentials, no brute-force protection, no input sanitization on file paths. Socket permissions (0660) are correct. Physical isolation helps but is not sufficient.
Documentation	4/10	Good README with architecture overview and test table. Arabic migration guide. But no inline API docs, no deployment docs beyond README, no architecture diagram.

Category	Score (1–10)	Justification
Reliability	6/10	Excellent auto-reconnect logic for both camera and serial hardware. Known runtime bugs (AttributeError, TypeError) need fixing. No watchdog process monitor. Boot self-test is a professional touch.

Overall Project Assessment:

This is a well-conceived, production-deployed industrial safety system built with clear engineering intent. The IPC architecture separating hardware from GUI, the pure OR-gate with zero internal timers, the comprehensive unit test suite, and the thoughtful role-based access control all demonstrate professional systems engineering.

The main weaknesses are in code hygiene (plaintext passwords, known runtime bugs, relative paths, no logging), the monolithic main file, and the confusion created by having two parallel codebases. For a safety-critical system, the plaintext password store and the potential runtime crashes are the highest-priority issues to address before further production deployment.

SECTION 25 — COMPLETE DATA FLOW

25.1 Camera Capture Data Path

```
graph TD
    A[USB Camera V4L2] -->|cv2.VideoCapture.read| B[numpy.ndarray BGR 640x480]
    B -->|Qt Signal frame_ready| C[CameraWorkerThread emit]
    C -->|Qt signal queue| D[Main Thread on_frame_received]
    D -->|every 3rd frame| E[YoloDetector.detect_persons]
    E -->|YOLO inference| F[persons list x1,y1,x2,y2]
    F -->|coordinate check| G[AlarmSystem.check_boundaries]
    G -->|bool warning| H{Person in ROI?}
    H -->|Yes| I[USBRelayManager.trigger_relay]
    H -->|No| J[Update GUI only]
    I -->|JSON IPC| K[hardware_worker SharedState]
    K -->|50ms poll| L[HardwareWorker OR-Gate]
    L -->|serial ASCII| M[RM04U Relay Board]
    M -->|contact closure| N[Physical Alarm Device]
    D -->|every frame| O[cv2.annotate frame]
    O -->|BGR to RGB| P[QImage]
    P -->|QPixmap| Q[VideoLabel display]
```

25.2 Configuration Data Path

```
[Operator draws ROI on VideoLabel]
    ↓ roi_updated signal (QRect in widget pixels)
[CameraCardWidget.save_roi()]
    ↓ json.load(config.json)
    ↓ modify data["camera_N"]
    ↓ json.dump(config.json)
[config.json] (persisted to disk)
    ↓ next startup: load_roi()
    ↓ json.load(config.json)
    ↓ QRect(x,y,w,h)
    ↓ video_label.set_roi(roi_rect)
    ↓ frame_received: scale to frame coords
    ↓ AlarmSystem.check_boundaries(scaled_roi)
```

25.3 Hardware Status Data Path

```
[hardware_worker.py]
serial connection succeeds
    ↓ SharedState.set("hw_status", "ONLINE")
    ↓ IPCServer.broadcast_status()
    ↓ [for each client socket]
        socket.sendall('{ "type": "status", "hw_status": "ONLINE" }\n')

[hardware_manager.py]
_receiver_loop() receives bytes
    ↓ json.loads(line)
    ↓ msg["type"] == "status"
    ↓ self._hw_status = "ONLINE" [under _hw_status_lock]

[ai_gui_system.py]
status_check_timer fires every 1 second
    ↓ poll_hardware_status()
    ↓ relay_manager.hw_status [property reads _hw_status under lock]
    ↓ is_online = (hw_status == "ONLINE")
    ↓ [if changed] self.hardware_is_online = is_online
    ↓ evaluate_global_alarms()
    ↓ update footer status banner
```

25.4 Security Data Path

```
[User selects role from dropdown]
    ↓ ForkliftSafetyGUI.change_role(role)
    ↓ QInputDialog.getText() → password string
    ↓ SecurityManager.verify_password(role, password)
    ↓ SecurityManager.load_passwords()
        ↓ json.load("security.json")
        ↓ dict {role: plaintext_password}
    ↓ passwords.get(role) == password → bool
    ↓ [True] self.current_role = role
    ↓ update_ui_permissions()
        ↓ role_level = access_levels[current_role]
        ↓ is_supervisor = role_level >= 3
        ↓ [enable/disable all gated UI widgets]
    ↓ [False] revert combo, show warning

[Supervisor changes password]
    ↓ change_password()
    ↓ SecurityManager.verify_password(role, current_password)
    ↓ SecurityManager.change_password(role, new_password)
        ↓ SecurityManager.load_passwords()
        ↓ passwords[role] = new_password
        ↓ json.dump(passwords, "security.json")
```

25.5 Serial Command Data Path (Relay Actuation)

```
HardwareWorker.transmit(channel=3, state_on=True)
↓
command = "N3" ("F3" if OFF)
ser.write(b"N3") [2 bytes to kernel serial buffer]
ser.flush()      [force drain to hardware]
↓
[USB-Serial CH340 converts to RS232-level serial signal]
↓
[RM04U UART receives at 9600 baud 8N1]
[RM04U parses command string]
[RM04U energizes relay coil 3]
↓
[Relay 3 NO contact closes]
↓
[External circuit (alarm/horn/PLC input) activates]
```

Timing breakdown:

- ser.write(): <1ms (kernel buffer write)
- ser.flush(): <1ms
- USB frame transmission at 9600 baud: 2 chars × 10 bits / 9600 = ~2.1ms
- CH340 to UART: <1ms
- RM04U relay coil energize: ~5-10ms (mechanical)
- TOTAL: ~10-15ms from transmit() call to relay closure

SECTION 26 — COMPLETE EVENT FLOW

26.1 Mouse Events — VideoLabel (roi.py)

Event	Source	Handler	Trigger Condition	Result
<code>mousePressEvent</code>	User click on VideoLabel	<code>VideoLabel.mousePressEvent()</code>	<code>is_editable=True</code>	Determines action: start draw, start drag, or start resize based on click location
<code>mouseMoveEvent</code>	Mouse drag	<code>VideoLabel.mouseMoveEvent()</code>	After press, while button held	Updates ROI rect: resizes if <code>resizing</code> , moves if <code>dragging</code> , draws new rect if <code>is_drawing</code>
<code>mouseReleaseEvent</code>	Mouse button release	<code>VideoLabel.mouseReleaseEvent()</code>	After any drag operation	Clears all state flags, emits <code>roi_updated(roi_rect)</code> signal
<code>mouseMoveEvent</code> (hover)	Mouse hover	<code>VideoLabel.mouseMoveEvent()</code>	No button held	Updates cursor shape: arrow/move/resize diagonal

26.2 Keyboard Events

Event	Source	Handler	Result
Any key press	User on main window	<code>ForkliftSafetyGUI.eventFilter()</code>	Resets <code>inactivity_timer</code> to 5 minutes
Alt+F4	OS window manager	<code>ForkliftSafetyGUI.closeEvent()</code>	If role < Supervisor: blocked with warning dialog. If Supervisor+: clean shutdown.

26.3 Qt Signals — Custom Application Signals

Signal	Class	Type	Emitted When	Connected To
<code>frame_ready(np.ndarray)</code>	<code>CameraWorkerThread</code>	<code>pyqtSignal(np.ndarray)</code>	Every successfully captured frame	<code>CameraCardWidget.on_frame_ready</code>
<code>status_changed(bool, str)</code>	<code>CameraWorkerThread</code>	<code>pyqtSignal(bool, str)</code>	Camera goes online or offline	<code>CameraCardWidget.on_status_changed</code>
<code>roi_updated(QRect)</code>	<code>VideoLabel</code>	<code>pyqtSignal(QRect)</code>	Mouse release after ROI draw/move/resize	<code>CameraCardWidget.on_roi_updated</code>

26.4 QTimer Events

Timer	Interval	Created In	Slot Connected	Parent
<code>flash_timer</code>	500ms	<code>CircularLED.__init__()</code>	<code>CircularLED.toggle_flash()</code>	<code>FlashLightWidget</code>
<code>blink_timer</code>	500ms	<code>CameraCardWidget.__init__()</code>	<code>CameraCardWidget.update_offline_banner()</code>	<code>CameraCardWidget</code>
<code>inactivity_timer</code>	300,000ms (5 min)	<code>ForkliftSafetyGUI.__init__()</code>	<code>ForkliftSafetyGUI.auto_logout()</code>	<code>ForkliftSafetyGUI</code>
<code>status_check_timer</code>	1,000ms	<code>ForkliftSafetyGUI.__init__()</code>	<code>ForkliftSafetyGUI.poll_hardware_status()</code>	<code>ForkliftSafetyGUI</code>

26.5 Camera Events

Event	Source	Handler	Result
Camera connect	V4L2 device becomes available	<code>CameraWorkerThread.run()</code> reconnect loop	<code>status_changed(True, "Online")</code> emitted
Camera disconnect	V4L2 device removed/error	<code>camera.cap.read()</code> returns <code>ret=False</code>	<code>camera.release()</code> , <code>camera=None</code> , 3s retry loop

Event	Source	Handler	Result
Frame read success	V4L2 kernel driver	<code>CameraWorkerThread.run()</code>	<code>frame_ready(frame)</code> emitted
Camera offline display	Camera is None	<code>on_camera_status_changed(False, ...)</code>	LED=error (amber), OFFLINE banner shown

26.6 Hardware Events

Event	Source	Detection	Handler	Result
Serial connect success	OS serial port open	<code>serial.Serial()</code> does not throw	<code>run_worker_thread()</code>	boot self-test runs, SharedState hw_status=ONLINE
Serial disconnect	Cable unplugged, device error	<code>serial.SerialException</code> in <code>HardwareWorker.run()</code>	<code>_teardown()</code> called	hw_status=OFFLINE, 3s reconnect sleep
Relay activation	OR-gate result changes 0 → 1	<code>targets[ch] != last_sent_state[ch]</code>	<code>transmit(ch, True)</code>	<code>ser.write(b"Nch")</code>
Relay deactivation	OR-gate result changes 1 → 0	<code>targets[ch] != last_sent_state[ch]</code>	<code>transmit(ch, False)</code>	<code>ser.write(b"Fch")</code>

26.7 IPC Socket Events

Event	Source	Detection	Handler	Result
Client connects	GUI <code>_connect_socket()</code>	<code>server_socket.accept()</code>	Spawn <code>_handle_client</code> thread	Add to <code>clients</code> list
Client sends command	GUI <code>_send_command()</code>	<code>conn.recv(4096)</code> returns data	<code>_process_command(cmd, conn)</code>	SharedState updated
Client disconnects	GUI process exit or <code>cleanup()</code>	<code>conn.recv()</code> returns empty bytes	<code>_handle_client</code> thread exits	Remove from <code>clients</code>
Server sends status	<code>broadcast_status()</code>	N/A (server-initiated)	All clients receive <code>{"type": "status", ...}</code>	GUI updates hw_status cache
IPC connection lost (client side)	Server restart / crash	<code>recv()</code> returns empty / <code>OSError</code>	<code>_receiver_loop</code> reconnects	<code>_hw_status = "OFFLINE"</code> , 2s sleep, <code>_connect_socket()</code>

26.8 Role Change Event Flow

```

User selects role in QComboBox
↓ QComboBox.currentTextChanged signal
↓ ForkliftSafetyGUI.change_role(role)
↓ [if role == "Operator"] → set role, update_ui_permissions(), return
↓ [else] QInputDialog.getText() → modal password dialog
↓ [user enters password] → ok=True
↓ SecurityManager.verify_password(role, password)
↓ json.load("security.json") → compare
↓ [match] self.current_role = role
    update_ui_permissions() → enable/disable widgets
    add_audit_log("Authentication success")
    QMessageBox.information()
↓ [no match] role_combo.setCurrentText(old_role) [silent revert]
    add_audit_log("Authentication Failure")
    QMessageBox.warning()

```

SECTION 27 — THREAD INTERACTION ANALYSIS

27.1 Thread Map

Process 1: ai_gui_system.py

```

Thread A: Qt Main Thread
[GUI rendering, Qt event loop, signal delivery, timer callbacks]
Accesses: all Qt widgets, relay_manager (via API), security, detector, cards
MUST NEVER BLOCK > few ms (blocks GUI responsiveness)

Thread B: CameraWorkerThread[0] (QThread, daemon=False)
Loop: read frame → emit frame_ready → sleep(0.03)
Accesses: Camera object (exclusive), device_index (read-only), running flag
Blocked on: cv2.VideoCapture.read() (~33ms)

Thread C: CameraWorkerThread[1,2,3] (identical to B, for cameras 2,3,4)

Thread D: USBRelayManager._recv_thread (daemon Thread)
Loop: sock.recv(4096) [1s timeout] → parse JSON → update_hw_status
Accesses: _socket (under _socket_lock), _hw_status (under _hw_status_lock)
Blocked on: sock.recv() with 1 second timeout

```

Process 2: hardware_worker.py

```

Thread E: Main thread (not really a thread – just main() then join/signal wait)

Thread F: IPCServer listener (daemon Thread)
Loop: server_socket.accept() [blocking] → spawn _handle_client thread
Accesses: server_socket (exclusive), clients (under clients_lock)

Thread G: IPCServer._handle_client (daemon Thread, one per client connection)
Loop: conn.recv(4096) → _process_command() → SharedState.set/update
Accesses: SharedState (via methods with _lock), clients (under clients_lock)

Thread H: run_worker_thread (daemon Thread)
Outer: serial reconnect loop
Inner: HardwareWorker.run() OR-gate loop (50ms cycle)
Accesses: SharedState (read via get_snapshot), ser (exclusive), ipc.broadcast

```


27.2 Shared Resource Access Matrix

Resource	Thread A (Main)	Thread B-E (Cam)	Thread D (Recv)	Thread G (IPC handler)	Thread H (Worker)
Qt Widgets	R/W (exclusive)	NEVER (signal only)	NEVER	NEVER	NEVER
<code>YoloDetector.model</code>	Creates/ Replaces	Reads (via slot)	NEVER	NEVER	NEVER
<code>USBRelayManager._socket</code>	Calls API methods	NEVER	R/W (under lock)	NEVER	NEVER
<code>USBRelayManager._hw_status</code>	Reads (under lock)	NEVER	Writes (under lock)	NEVER	NEVER
<code>SharedState._data</code>	NEVER	NEVER	NEVER	Writes (under lock)	Reads snapshot (under lock)
<code>CameraWorkerThread.running</code>	Writes (stop())	Reads (loop)	NEVER	NEVER	NEVER
<code>IPCServer.clients</code>	NEVER	NEVER	NEVER	R/W (under lock)	Reads (broadcast, under lock)
<code>serial.Serial</code>	NEVER (indirect via IPC)	NEVER	NEVER	NEVER	R/W (exclusive)
<code>config.json</code>	R/W (main thread)	NEVER	NEVER	Reads at startup	Reads at startup
<code>security.json</code>	R/W (main thread)	NEVER	NEVER	NEVER	NEVER

27.3 Race Condition Analysis

Scenario	Threads Involved	Risk Level	Mitigation
<code>CameraWorkerThread.running</code> set False while run() reads it	Main + Camera thread	LOW	CPython GIL makes bool assignment atomic; no partial-write possible
<code>SharedState._data</code> read (worker) while write (IPC handler)	Thread G + Thread H	MITIGATED	<code>threading.Lock()</code> around all access; <code>get_snapshot()</code> copies under lock
<code>USBRelayManager._socket</code> read (recv thread) + write (main thread via send)	Thread A + Thread D	MITIGATED	<code>_socket_lock</code> guards all access
<code>USBRelayManager._hw_status</code> read (main, status poll) + write (recv thread)	Thread A + Thread D	MITIGATED	<code>_hw_status_lock</code> guards all access
Qt widget update from non-main thread	Any non-main thread	CRITICAL RISK	By design, Qt widgets are only updated from Qt signal slots (marshalled to main thread). No direct Qt calls from background threads exist.
<code>camera</code> object in CameraWorkerThread accessed from main thread	Thread A (stop_camera) + Thread B	LOW	<code>stop()</code> sets <code>running=False</code> and waits; Camera is only released after run() exits

Scenario	Threads Involved	Risk Level	Mitigation
Multiple relay trigger calls racing on <code>relay_states</code>	Main thread only	NONE	All relay calls occur in Qt signal slots which are serialized on the main thread
<code>clients</code> list modified (add in F, remove in H via broadcast)	Thread F + Thread H	MITIGATED	<code>clients_lock</code> guards both add and remove operations

27.4 Deadlock Risk Analysis

Scenario	Locks Involved	Risk
<code>_socket_lock</code> then <code>_hw_status_lock</code>	In <code>_receiver_loop</code> : <code>_socket_lock</code> released before <code>_hw_status_lock</code> acquired — no nested holding	NONE
<code>SharedState._lock</code> held during <code>get_snapshot()</code>	Only one lock, never nested	NONE
<code>clients_lock</code> held during <code>broadcast_status()</code>	<code>clients_lock</code> held while calling <code>conn.sendall()</code> . If send blocks, all new client additions are blocked.	LOW RISK — Unix Domain Socket sends are fast; mitigation: set socket non-blocking
Qt main thread calls <code>thread.wait(3000)</code> inside slot	If camera thread emits signal while main thread is waiting, Qt would deadlock	MITIGATED — Qt's <code>wait()</code> inside a slot processes events; signal delivery may still occur

27.5 Thread Shutdown Sequence

```

ForkliftSafetyGUI.closeEvent() [main thread]
↓
[for each card] CameraCardWidget.stop_camera()
  ↓ camera_thread.stop():
    camera_thread.running = False [signals thread to exit]
    camera_thread.wait(3000)      [main thread blocks up to 3s]
  ↓ [CameraWorkerThread.run() sees running=False, exits loop]
  ↓ Camera.release() called in thread
  ↓ thread.wait() returns
  ↓ camera_thread.deleteLater() [Qt deferred delete]
↓
relay_manager.disconnect() [sends disconnect command via socket]
relay_manager.cleanup():
  ↓ _recv_running = False [signals recv thread to stop]
  ↓ _socket.shutdown(SHUT_RDWR) [unblocks recv() in recv thread]
  ↓ _socket.close()
  [recv thread: recv() returns error, loop exits, thread ends]
↓
event.accept() [Qt destroys window]
↓
QApplication.exit()
↓
Python runtime exits
[All daemon threads killed by OS]

```

SECTION 28 — DEPENDENCY MATRIX

28.1 File Import Dependency Matrix

File	Imports From	Imported By	Dependency Direction	Circular?
<code>ai_gui_system.py</code>	<code>security</code> , <code>camera</code> , <code>detector</code> , <code>alarm</code> , <code>roi</code> , <code>hardware_manager</code> , <code>PyQt5</code> , <code>cv2</code> , <code>numpy</code> , <code>json</code> , <code>sys</code> , <code>time</code> , <code>os</code>	Entry point — nobody imports this	Downstream only	No
<code>hardware_worker.py</code>	<code>serial</code> , <code>socket</code> , <code>threading</code> , <code>json</code> , <code>time</code> , <code>glob</code> , <code>os</code> , <code>sys</code> , <code>fcntl</code>	<code>test_or_gate_logic.py</code> (dynamic import)	Standalone	No
<code>hardware_manager.py</code>	<code>socket</code> , <code>threading</code> , <code>json</code> , <code>time</code> , <code>glob</code> , <code>os</code> , <code>serial.tools.list_ports</code> (optional)	<code>ai_gui_system.py</code>	Upstream	No
<code>detector.py</code>	<code>ultralytics.YOLO</code> , <code>os</code>	<code>ai_gui_system.py</code> , <code>forklift_ai_safety/</code> <code>core/camera_thread.py</code>	Upstream	No
<code>camera.py</code>	<code>cv2</code>	<code>ai_gui_system.py</code> , <code>forklift_ai_safety/</code> <code>core/camera_thread.py</code>	Upstream	No
<code>alarm.py</code>	<code>PyQt5.QtCore</code> (<code>QPoint</code> , <code>QRect</code>)	<code>ai_gui_system.py</code>	Upstream	No
<code>roi.py</code>	<code>PyQt5</code> (<code>QLabel</code> , <code>QPainter</code> , <code>QPen</code> , etc.)	<code>ai_gui_system.py</code>	Upstream	No
<code>security.py</code>	<code>os</code> , <code>json</code>	<code>ai_gui_system.py</code> , <code>forklift_ai_safety/</code> <code>core/security.py</code>	Upstream	No
<code>test_or_gate_logic.py</code>	<code>hardware_worker.py</code> (dynamic), <code>unittest</code> , <code>threading</code> , <code>time</code>	Nothing	Test only	No
<code>forklift_ai_safety/</code> <code>main.py</code>	<code>PyQt5</code> , <code>ui.main_window</code>	Entry point	Downstream only	No
<code>forklift_ai_safety/ui/</code> <code>main_window.py</code>	<code>ui.camera_panel</code> , <code>ui.sidebar_panel</code> , <code>ui.status_bar</code> , <code>core.security</code> , <code>core.roi_manager</code> , <code>core.camera_thread</code>	<code>main.py</code>	Upstream	No
<code>forklift_ai_safety/</code> <code>core/camera_thread.py</code>	<code>core.camera</code> , <code>core.detector</code> , <code>core.alarm_manager</code> , <code>PyQt5</code>	<code>ui.main_window</code>	Upstream	No

File	Imports From	Imported By	Dependency Direction	Circular?
<code>forklift_ai_safety/core/camera.py</code>	<code>cv2</code> , <code>numpy</code> , <code>time</code> , <code>random</code>	<code>core.camera_thread</code>	Upstream	No
<code>forklift_ai_safety/core/detector.py</code>	<code>ultralytics.YOLO</code> , <code>os</code>	<code>core.camera_thread</code>	Upstream	No
<code>forklift_ai_safety/core/alarm_manager.py</code>	<code>PyQt5.QtCore</code>	<code>core.camera_thread</code>	Upstream	No
<code>forklift_ai_safety/core/roi_manager.py</code>	<code>os</code> , <code>json</code> , <code>PyQt5.QtCore</code>	<code>ui.main_window</code>	Upstream	No
<code>forklift_ai_safety/core/security.py</code>	<code>os</code> , <code>json</code>	<code>ui.main_window</code>	Upstream	No

28.2 External Package Dependency Analysis

Package	Version in requirements.txt	Used By	What Breaks If Missing
<code>PyQt5</code>	Not pinned (system package)	<code>ai_gui_system.py</code> , <code>roi.py</code> , <code>alarm.py</code> , all <code>forklift_ai_safety/ui/</code>	Entire GUI fails to start
<code>ultralytics</code>	Not in requirements.txt (must pip install)	<code>detector.py</code>	YoloDetector cannot load; system starts but no detection
<code>opencv-python</code> (<code>cv2</code>)	Not in requirements.txt (must pip install)	<code>camera.py</code> , all <code>camera_thread.py</code>	No camera capture, no frame processing
<code>pyserial</code> (<code>serial</code>)	<code>pyserial==3.5</code>	<code>hardware_worker.py</code>	Worker exits immediately with FATAL error
<code>numpy</code>	Not pinned (transitive from <code>cv2</code>)	<code>ai_gui_system.py</code> , <code>camera_thread.py</code>	Frame handling breaks
<code>torch</code>	Not in requirements.txt (transitive from <code>ultralytics</code>)	YOLO backend	YOLO inference fails

Critical Gap: `requirements.txt` contains system-level Ubuntu packages, not the actual Python packages needed. The actual Python packages needed are: `PyQt5`, `opencv-python`, `ultralytics`, `pyserial`, `numpy`. These are not in `requirements.txt` and must be installed manually or via `setup_linux.sh`.

28.3 Unused Dependencies

Import	File	Usage	Status
<code>torch</code>	<code>ai_gui_system.py</code> (try/except)	Not used directly — import-only for DLL conflict workaround	Used — the import forces PyTorch to load before Qt on Windows
<code>glob</code>	<code>hardware_manager.py</code>	<code>scan_available_ports()</code>	Used

Import	File	Usage	Status
<code>QFileDialog</code>	<code>ai_gui_system.py</code>	Browse model file in Config page	Used
<code>get_status_styles()</code> in <code>alarm.py</code>	<code>alarm.py</code>	Not called from <code>ai_gui_system.py</code>	Unused in production
<code>get_camera_error_styles()</code> in <code>alarm.py</code>	<code>alarm.py</code>	Not called from <code>ai_gui_system.py</code>	Unused in production
<code>HAS_SERIAL</code> flag	<code>hardware_manager.py</code>	Never checked after assignment	Dead code
<code>DEFAULT_ON_DELAY</code> , <code>DEFAULT_OFF_DELAY</code>	<code>hardware_manager.py</code>	Set as instance attributes, never used in delay logic	Effectively unused

SECTION 29 — CONFIGURATION USAGE MAP

29.1 Complete Configuration Value Usage

Config Key	Path in JSON	Default	Loaded In
<code>camera_0.x</code>	<code>config.json["camera_0"] ["x"]</code>	0	<code>CameraCardWidget.load_roi()</code>
<code>camera_0.y</code>	<code>config.json["camera_0"] ["y"]</code>	184	<code>CameraCardWidget.load_roi()</code>
<code>camera_0.w</code>	<code>config.json["camera_0"] ["w"]</code>	723	<code>CameraCardWidget.load_roi()</code>
<code>camera_0.h</code>	<code>config.json["camera_0"] ["h"]</code>	144	<code>CameraCardWidget.load_roi()</code>
<code>settings.com_port</code>	<code>config.json["settings"] ["com_port"]</code>	<code>/dev/ ttyUSB0</code>	<code>ForkliftSafetyGUI.load_global_set</code> <code>hardware_worker.get_config_port()</code>
<code>settings.baud_rate</code>	<code>config.json["settings"] ["baud_rate"]</code>	9600	<code>load_global_settings()</code>

Config Key	Path in JSON	Default	Loaded In
<code>settings.camera_mapping</code>	<code>config.json["settings"] ["camera_mapping"]</code>	<code>[0,2,4,6]</code>	<code>load_global_settings()</code>
<code>settings.relay_mapping</code>	<code>config.json["settings"] ["relay_mapping"]</code>	<code>[3,2,3,2]</code>	<code>load_global_settings()</code> , <code>load_initial_config()</code> (hw worker)
<code>settings.model_path</code>	<code>config.json["settings"] ["model_path"]</code>	<code>yolov8n.pt</code>	<code>load_global_settings()</code>
<code>settings.confidence</code>	<code>config.json["settings"] ["confidence"]</code>	<code>0.35</code>	<code>load_global_settings()</code>
<code>settings.on_delay</code>	<code>config.json["settings"] ["on_delay"]</code>	<code>0.0</code>	<code>load_global_settings()</code>
<code>settings.off_delay</code>	<code>config.json["settings"] ["off_delay"]</code>	<code>0.1</code>	<code>load_global_settings()</code>
<code>settings.boot_self_test</code>	<code>config.json["settings"] ["boot_self_test"]</code>	<code>1</code>	<code>load_global_settings()</code> , <code>load_initial_config()</code>
<code>settings.hardware_coil_enabled</code>	<code>config.json["settings"] ["hardware_coil_enabled"]</code>	<code>1</code>	<code>load_global_settings()</code> , <code>load_initial_config()</code>

29.2 Configuration Validation Gaps

Config Value	Validation Present	Risk of Bad Value	Recommended Validation
<code>com_port</code>	None	Wrong port — serial open fails, auto-retry	Validate <code>/dev/</code> prefix on Linux
<code>baud_rate</code>	None	Wrong baud = relay board unresponsive	Only allow: 9600, 19200, 38400, 115200
<code>camera_mapping</code>	Length checked (4 slots)	Index out of range for V4L2	Validate each is string path or int 0-20
<code>relay_mapping</code>	Length padded to 4	Value outside 1-4 causes Phase 2 to never match	Validate each value in range 1-4
<code>confidence</code>	Slider range 0.25-0.90 enforced in UI	Below 0 or above 1.0 crashes YOLO	Clamp to [0.0, 1.0] before passing
<code>model_path</code>	<code>os.path.exists()</code> checked at load	Missing file triggers auto-download	Check before YOLO() call
ROI coordinates	None	x+w or y+h exceeds frame dims	Clamp to frame dimensions on load

SECTION 30 — MEMORY OWNERSHIP ANALYSIS

30.1 Python Memory Model in This Project

Python uses reference counting with cycle detection via its garbage collector. Objects are freed when their reference count drops to 0. There is no explicit `free()` call — developers must ensure no lingering references prevent deallocation.

30.2 Object Reference Chain Analysis

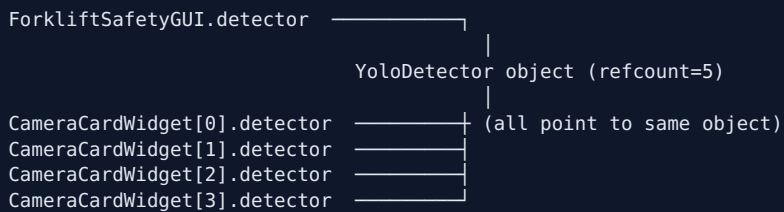
```

QApplication
├── (holds strong reference to all QWidget children via Qt object tree)
│   └── ForkliftSafetyGUI
│       ├── (Python attribute) self.detector → YoloDetector
│       │   └── self.model → ultralytics.YOLO [heavy object: ~80MB RAM]
│       ├── (Python attribute) self.relay_manager → USBRelayManager
│       │   └── self._socket → socket.socket [OS file descriptor]
│       ├── (Python attribute) self.security → SecurityManager [lightweight]
│       ├── (Qt parent) self.cards[0..3] → CameraCardWidget x4
│       │   ├── (Python attribute) self.detector → SAME YoloDetector [shared ref]
│       │   ├── (Python attribute) self.relay_manager → SAME USBRelayManager [shared ref]
│       │   ├── (Python attribute) self.alarm_system → AlarmSystem [lightweight]
│       │   ├── (Qt parent) self.video_label → VideoLabel
│       │   ├── (Qt parent) self.led → CircularLED
│       │   └── (Python attribute) self.camera_thread → CameraWorkerThread (or None)
│       │       ├── self.camera → Camera
│       │       └── self.cap → cv2.VideoCapture [OS file descriptor to /dev/videoN]
│       ├── (Qt parent) self.inactivity_timer → QTimer
│       └── (Qt parent) self.status_check_timer → QTimer

```

Key Ownership Rule: Qt owns its widget children (via Qt's parent-child tree). Python `self.` attributes hold additional Python references. Both must release for an object to be freed.

30.3 YoloDetector Shared Reference Danger



When the model is changed, `save_global_configurations()` does:

```

self.detector = YoloDetector(new_path, new_conf) # old refcount drops by 1
for card in self.cards:
    card.detector = self.detector                # 4 more drops
  
```

Old `YoloDetector` refcount goes 5 → 0 and is freed. This is correct. **Risk:** If YOLO model fails to load (exception in `YoloDetector.__init__`), `self.detector` may be left as the new broken object and all cards get it. Should be guarded with try/except that reverts to old detector on failure.

30.4 Socket File Descriptor Ownership

FD Owner	Created	Closed	OS cleanup if not closed
<code>USBRelayManager._socket</code>	<code>socket.socket()</code> in <code>_connect_socket()</code>	<code>cleanup()</code> calls <code>_socket.close()</code>	OS closes on process exit; socket file remains on filesystem
<code>IPCTServer.server_socket</code>	<code>socket.socket()</code> in <code>_init__()</code>	Never explicitly	OS closes on process exit; <code>/dev/shm/forklift_relay.sock</code> file remains
<code>IPCTServer.clients[i]</code>	<code>server_socket.accept()</code>	<code>conn.close()</code> in <code>_handle_client</code> on disconnect	OS closes on process exit
<code>cv2.VideoCapture</code>	<code>cv2.VideoCapture(idcx)</code> in <code>Camera.initialize_camera()</code>	<code>cap.release()</code> in <code>Camera.release()</code>	V4L2 device released by OS on process exit; other processes can then use it
<code>serial.Serial</code>	<code>serial.Serial(port, baud)</code> in <code>open_serial_connection()</code>	<code>ser.close()</code> in <code>_teardown()</code>	OS closes on process exit; DTR/RTS lines may stay asserted depending on CH340 state
<code>fcntl lock_fd</code>	<code>open(LOCK_PATH)</code> in <code>check_singleton()</code>	Process exit	OS releases flock automatically on process death

End of PROJECT_FULL_DOCUMENTATION.md Document generated: 2026-07-16 — AI_Forklift_Safety Project

PART II — ENGINEERING REFERENCE MANUAL

EXTENSION

This section extends the base documentation with deep reverse-engineering analysis, variable cross-references, complete call graphs, state machine diagrams, and all engineering details required for long-term maintenance without access to the original developer.

SECTION 21 — COMPLETE VARIABLE REFERENCE

21.1 Module-Level Constants — hardware_worker.py

Variable	Type	Value	Scope	Purpose	Re
SCRIPT_DIR	str	os.path.dirname(os.path.abspath(__file__))	Module global	Absolute path to the directory containing hardware_worker.py . Used to resolve config.json regardless of CWD.	g L
SOCKET_PATH	str	"/dev/shm/forklift_relay.sock"	Module global	Path to the Unix Domain Socket file on tmpfs. Both processes must agree on this exact path.	I U
LOCK_PATH	str	"/tmp/hardware_worker.lock"	Module global	Path to the singleton flock file. Prevents duplicate daemon instances.	c
UDEV_RELAY_PATH	str	"/dev/forklift_relay"	Module global	udev symlink for the RM04U relay board. Priority 1 in serial port resolution.	g
BAUD_RATE	int	9600	Module global	Serial communication speed. Must match the RM04U board's hardware configuration.	o
SERIAL_TIMEOUT	int	1	Module global	pyserial timeout parameter in seconds. Governs ser.read() blocking duration.	o
POLL_INTERVAL	float	0.05	Module global	OR-gate loop sleep duration (50ms). Controls maximum relay response latency.	H

Variable	Type	Value	Scope	Purpose	Risk
<code>RECONNECT_INTERVAL</code>	<code>float</code>	<code>3.0</code>	Module global	Seconds to wait between serial reconnect attempts.	n
<code>SELF_TEST_PULSE</code>	<code>float</code>	<code>0.3</code>	Module global	Duration in seconds each relay stays ON during boot self-test.	n

Risk Note: `POLL_INTERVAL = 0.05` means worst-case relay response is 50ms after the SharedState is updated. Increasing this value increases system latency. Decreasing it increases CPU wakeup frequency. The current value balances safety response time against CPU load.

21.2 Module-Level Constants — `hardware_manager.py`

Variable	Type	Value	Scope	Purpose	Risk
<code>SOCKET_PATH</code>	<code>str</code>	<code>"/dev/shm/forklift_relay.sock"</code>	Module global	Must match <code>hardware_worker.py</code> exactly. Mismatch = silent failure (connection refused).	Path mismatch breaks the entire IPC channel
<code>DEFAULT_ON_DELAY</code>	<code>float</code>	<code>0.2</code>	Module global	Default debounce delay applied to <code>USBRelayManager.on_delay</code> .	Currently unused by logic (see Known Issues)
<code>DEFAULT_OFF_DELAY</code>	<code>float</code>	<code>1.5</code>	Module global	Default hold delay applied to <code>USBRelayManager.off_delay</code> .	Currently unused by logic
<code>HAS_SERIAL</code>	<code>bool</code>	<code>True / False</code>	Module global	Set to True if <code>serial.tools.list_ports</code> imports successfully. Guards port-scan code paths.	If False, port scan always returns defaults

21.3 `SharedState.data` — Complete Key Reference

Key	Type	Default	Writable By	Read By	S
<code>station_1_trigger</code>	<code>int</code> (0/1)	<code>0</code>	<code>IPCTServer.process_command</code> (cmd=trigger, station=1)	<code>HardwareWorker.run()</code> Phase 1	1 h p
<code>station_2_trigger</code>	<code>int</code> (0/1)	<code>0</code>	Same	Same	1
<code>station_3_trigger</code>	<code>int</code> (0/1)	<code>0</code>	Same	Same	1

Key	Type	Default	Writable By	Read By	S
<code>station_4_trigger</code>	<code>int</code> (0/1)	<code>0</code>	Same	Same	1
<code>station_1_relay</code>	<code>int</code> (1–4)	From config	<code>IPCServer._process_command</code> (cmd=relay_mapping)	<code>HardwareWorker.run()</code> Phase 2	P cl 1
<code>station_2_relay</code>	<code>int</code> (1–4)	From config	Same	Same	P cl
<code>station_3_relay</code>	<code>int</code> (1–4)	From config	Same	Same	P cl
<code>station_4_relay</code>	<code>int</code> (1–4)	From config	Same	Same	P cl
<code>manual_test</code>	<code>int</code> (0/1)	<code>0</code>	<code>IPCServer._process_command</code> (cmd>manual_test)	<code>HardwareWorker.run()</code> Phase 2	1 m (
<code>boot_self_test</code>	<code>int</code> (0/1)	From config	<code>IPCServer._process_command</code> (cmd=config)	<code>run_worker_thread()</code>	1 te co
<code>hardware_coil_enabled</code>	<code>int</code> (0/1)	From config	<code>IPCServer._process_command</code> (cmd=config)	<code>HardwareWorker.run()</code> Phase 2	0 ta on
<code>hw_status</code>	<code>str</code>	<code>"OFFLINE"</code>	<code>SharedState.set_hw_status()</code>	<code>IPCServer.broadcast_status()</code>	C co

21.4 `HardwareWorker` Instance Variables

Variable	Type	Default	Created In	Modified In	Read In	P
<code>ser</code>	<code>serial.Serial</code>	Passed from <code>run_worker_thread()</code>	<code>__init__</code>	Never (reference)	<code>transmit()</code> , <code>run()</code>	C p fo re
<code>state</code>	<code>SharedState</code>	Passed from <code>main()</code>	<code>__init__</code>	Never (reference)	<code>run()</code> (via <code>get_snapshot()</code>)	R to st
<code>ipc</code>	<code>IPCServer</code>	Passed from <code>main()</code>	<code>__init__</code>	Never (reference)	<code>run()</code> (via <code>broadcast_status()</code>)	R to st st b

Variable	Type	Default	Created In	Modified In	Read In	P
<code>last_sent_state</code>	<code>dict[int, None/int]</code>	<code>{1: None, 2: None, 3: None, 4: None}</code>	<code>__init__</code>	<code>run()</code> via <code>transmit()</code>	<code>run()</code> Phase 3 comparison	T s tr re I n (t s fi

Critical Detail on `last_sent_state` Initialization: The initial value `None` is intentional. On the very first OR-gate cycle, `targets[ch] = 0` (all stations are OFF). Since `0 != None`, the worker transmits `F1 F2 F3 F4` — explicitly resetting all relays to OFF at boot. This prevents relays from being left in an unknown state if a previous session terminated unexpectedly.

21.5 `CameraWorkerThread` Instance Variables

Variable	Type	Default	Created	Modified	Read	Thread Safety	Purpose
<code>device_index</code>	<code>int</code> or <code>str</code>	From constructor	<code>__init__</code>	<code>__init__</code> (type cast only)	<code>run()</code> → <code>Camera()</code>	Safe — set before thread starts	V4L2 device node or path for this camera station
<code>width</code>	<code>int</code>	<code>640</code>	<code>__init__</code>	Never	<code>run()</code> → <code>Camera()</code>	Safe	Target capture width
<code>height</code>	<code>int</code>	<code>480</code>	<code>__init__</code>	Never	<code>run()</code> → <code>Camera()</code>	Safe	Target capture height

Variable	Type	Default	Created	Modified	Read	Thread Safety	Purpose
<code>running</code>	<code>bool</code>	<code>False</code>	<code>__init__</code>	<code>run()</code> sets True; <code>stop()</code> sets False	<code>run()</code> loop condition	Not thread-safe — Qt sets it from main thread, worker reads it. Python bool assignment is atomic on CPython due to GIL, making this safe in practice.	Loop control flag
<code>camera</code>	<code>Camera</code> or <code>None</code>	<code>None</code>	<code>__init__</code>	<code>run()</code> (create/release on connect/disconnect)	<code>run()</code> (read_frame)	Not thread-safe — only accessed from worker thread	Current camera capture object
<code>last_reconnect_time</code>	<code>float</code>	<code>0</code> (local variable in <code>run()</code>)	First iteration of <code>run()</code>	Each reconnect attempt	Each frame loop iteration	Not thread-safe — local to <code>run()</code>	Throttles reconnect attempts to max once per 3 seconds

21.6 `CameraCardWidget` Instance Variables

Variable	Type	Default	Where Set	Thread Safety
<code>cam_id</code>	<code>int</code> (0–3)	Constructor	<code>__init__</code>	Safe — never modified
<code>device_index</code>	<code>int</code> or <code>str</code>	From <code>ForkliftSafetyGUI</code>	<code>__init__</code>	Safe — modified only from main thread
<code>detector</code>	<code>YoloDetector</code>	Shared from parent	<code>__init__</code>	Safe — <code>detect_person</code> is GIL-protected
<code>security</code>	<code>SecurityManager</code>	Shared from parent	<code>__init__</code>	Safe — file reads are atomic

Variable	Type	Default	Where Set	Thread Safety
<code>relay_manager</code>	<code>USBRelayManager</code>	Shared from parent	<code>__init__</code>	Safe — <code>relay_manager</code> methods are interrupt-safe and thread-safe
<code>alarm_system</code>	<code>AlarmSystem</code>	<code>AlarmSystem()</code>	<code>__init__</code>	Safe — pure computation, no shared state
<code>video_label</code>	<code>VideoLabel</code>	<code>VideoLabel()</code>	<code>__init__</code>	Safe — Qt widgets are main-thread only
<code>led</code>	<code>CircularLED</code>	<code>CircularLED()</code>	<code>__init__</code>	Safe — Qt widgets are main-thread only
<code>camera_thread</code>	<code>CameraWorkerThread</code> or <code>None</code>	<code>None</code>	<code>start_camera()</code> / <code>stop_camera()</code>	Safe — only modified from main thread
<code>camera_online</code>	<code>bool</code>	<code>False</code>	<code>on_camera_status_changed()</code>	Safe — Qt signal delivery is main-thread only
<code>is_active</code>	<code>bool</code>	<code>False</code>	<code>start_camera()</code> / <code>stop_camera()</code>	Safe — main thread only
<code>blink_timer</code>	<code>QTimer</code>	<code>QTimer()</code>	<code>__init__</code>	Safe — Qt timers are on main thread
<code>prev_warning</code>	<code>bool</code>	<code>False</code>	<code>__init__</code>	Safe — main thread only
<code>last_hardware_state</code>	<code>bool</code> or <code>None</code>	<code>None</code>	<code>__init__</code> / reset in override handler	Safe — main thread only
<code>frame_counter</code>	<code>int</code>	<code>0</code>	<code>__init__</code>	Safe — main thread only (signal delivery)

Variable	Type	Default	Where Set	Thread Safety
<code>last_persons</code>	<code>list</code>	(via <code>getattr</code>)	Set on first YOLO inference	Safe — main thread only
<code>last_warning</code>	<code>bool</code>	(via <code>getattr</code>)	Set on first YOLO inference	Safe — main thread only
<code>last_scaled_roi</code>	<code>QRect</code>	(via <code>getattr</code>)	Set on first YOLO inference	Safe — main thread only
<code>was_overridden</code>	<code>bool</code>	<code>False</code>	Frame handler when override detected	Safe — main thread only

21.7 `USBRelayManager` Instance Variables (Complete)

Variable	Type	Default	Lock Required	Purpose
<code>port_name</code>	<code>str</code> or <code>None</code>	<code>None</code>	None (main thread only)	Last recorded serial port name (informational only in IPC mode)
<code>is_connected</code>	<code>bool</code>	<code>False</code>	None (main thread only)	GUI's view of connection state
<code>relay_states</code>	<code>list[bool]×4</code>	<code>[False, False, False, False]</code>	None (main thread only)	Last-sent state per relay channel (latch for duplicate suppression)
<code>last_cmd_time</code>	<code>list[float]×4</code>	<code>[0.0, 0.0, 0.0, 0.0]</code>	None (main thread only)	Timestamp of last ON command per channel (500ms cooldown guard)
<code>on_delay</code>	<code>float</code>	<code>0.2</code>	None	Stored but not acted on (see Known Issues)
<code>off_delay</code>	<code>float</code>	<code>1.5</code>	None	Stored but not acted on (see Known Issues)
<code>boot_self_test</code>	<code>int</code>	<code>1</code>	None	Sent to server on <code>connect_port()</code>
<code>hardware_coil_enabled</code>	<code>int</code>	<code>1</code>	None	Mirrors server-side coil enable state

Variable	Type	Default	Lock Required	Purpose
<code>_socket</code>	<code>socket.socket</code> or <code>None</code>	<code>None</code>	<code>_socket_lock</code>	Active Unix Domain Socket connection
<code>_socket_lock</code>	<code>threading.Lock</code>	<code>Lock()</code>	N/A	Guards <code>_socket</code> access from multiple threads
<code>_recv_thread</code>	<code>Thread</code> or <code>None</code>	<code>None</code>	None	Background receiver thread reference
<code>_recv_running</code>	<code>bool</code>	<code>True</code>	None	Controls receiver loop and reconnect loop lifetime
<code>_hw_status</code>	<code>str</code>	<code>"OFFLINE"</code>	<code>_hw_status_lock</code>	Cached hardware status received from server pushes
<code>_hw_status_lock</code>	<code>threading.Lock</code>	<code>Lock()</code>	N/A	Guards <code>_hw_status</code> across main thread reads and recv thread writes

21.8 ForkliftSafetyGUI Instance Variables (Key Selection)

Variable	Type	Default	Purpose	Thread Safety
<code>current_role</code>	<code>str</code>	<code>"Operator"</code>	Active access level role name	Main thread only
<code>access_levels</code>	<code>dict[str,int]</code>	Hard-coded dict	Maps role names to numeric levels	Main thread only
<code>security</code>	<code>SecurityManager</code>	<code>SecurityManager()</code>	Authentication handler	Main thread only (file I/O)
<code>global_settings</code>	<code>dict</code>	Loaded from <code>config.json</code>	All runtime configuration values	Main thread only
<code>detector</code>	<code>YoloDetector</code>	<code>YoloDetector(...)</code>	Shared YOLO model	Main thread creates; shared read-only to cards
<code>relay_manager</code>	<code>USBRelayManager</code>	<code>USBRelayManager()</code>	Shared hardware IPC client	Thread-safe (internal locks)
<code>cards</code>	<code>list[CameraCardWidget]</code>	<code>[]</code> then 4 cards	All 4 station widgets	Main thread only

Variable	Type	Default	Purpose	Thread Safety
<code>stacked_widget</code>	<code>QStackedWidget</code>	<code>QStackedWidget()</code>	Page navigation (Live/Config)	Main thread only
<code>inactivity_timer</code>	<code>QTimer</code>	<code>QTimer()</code>	5-minute auto-logout	Main thread only (Qt timer)
<code>status_check_timer</code>	<code>QTimer</code>	<code>QTimer()</code>	1-second hardware status poll	Main thread only (Qt timer)
<code>hardware_is_online</code>	<code>bool</code>	<code>True</code>	Cached relay online state	Main thread only
<code>hardware_offline_blink</code>	<code>bool</code>	<code>False</code>	Alternates for blink animation on offline banner	Main thread only
<code>log_terminal</code>	<code>QListWidget</code>	<code>QListWidget()</code>	Audit log display (50-entry ring buffer)	Main thread only
<code>role_combo</code>	<code>QComboBox</code>	<code>QComboBox()</code>	Role selection dropdown	Main thread only

21.9 Complete Variable Cross-Reference Table

Variable	Defined In	First Consumed In	Flows To
Camera frame (numpy.ndarray)	<code>CameraWorkerThread.run()</code> → <code>camera.read_frame()</code>	<code>CameraCardWidget.on_frame_received()</code> via Qt signal	YOLO inference display
<code>warning</code> (bool)	<code>AlarmSystem.check_boundaries()</code>	<code>CameraCardWidget.on_frame_received()</code>	<code>USBRelayManager.send_command()</code> + GUI LED + siren
<code>persons</code> (list of tuples)	<code>YoloDetector.detect_persons()</code>	<code>AlarmSystem.check_boundaries()</code>	Frame annotations
<code>SharedState._data</code> (dict)	<code>SharedState.__init__()</code>	<code>HardwareWorker.run()</code>	Serial command queue
<code>targets</code> (dict)	<code>HardwareWorker.run()</code> Phase 2	<code>HardwareWorker.run()</code> Phase 3	<code>transmit()</code>
<code>config.json</code> content	<code>load_global_settings()</code>	<code>ForkliftSafetyGUI.__init__()</code>	All configuration
<code>security.json</code> content	<code>SecurityManager.load_passwords()</code>	<code>SecurityManager.verify_password()</code>	Password comparison
IPC JSON message	<code>USBRelayManager._send_command()</code>	<code>IPCServer._handle_client()</code>	<code>SharedState._data</code>

Variable	Defined In	First Consumed In	Flows To
<code>relay_states[i]</code> (bool)	<code>USBRelayManager.__init__()</code>	<code>trigger_relay()</code> latch check	Prevents duplic
<code>last_sent_state[ch]</code> (int/None)	<code>HardwareWorker.__init__()</code>	<code>HardwareWorker.run()</code> Phase 3	Controls <code>tran</code>

SECTION 22 — OBJECT LIFETIME ANALYSIS

22.1 Object Ownership Tree — Process 1 (ai_gui_system.py)

```
QApplication (app)
├── ForkliftSafetyGUI (window) [Owner: QApplication]
│   ├── SecurityManager (self.security) [Owner: ForkliftSafetyGUI]
│   ├── YoloDetector (self.detector) [Owner: ForkliftSafetyGUI]
│   │   ├── YOLO model (self.model) [Owner: YoloDetector]
│   │   │   ├── PyTorch tensors in RAM [Owner: ultralytics/PyTorch]
│   ├── USBRelayManager (self.relay_manager) [Owner: ForkliftSafetyGUI]
│   │   ├── socket.socket (self._socket) [Owner: USBRelayManager]
│   │   ├── threading.Lock (_socket_lock) [Owner: USBRelayManager]
│   │   ├── threading.Lock (_hw_status_lock) [Owner: USBRelayManager]
│   │   └── Thread (_recv_thread) [Owner: USBRelayManager (daemon)]
│   ├── QTimer (inactivity_timer) [Owner: ForkliftSafetyGUI (Qt parent)]
│   ├── QTimer (status_check_timer) [Owner: ForkliftSafetyGUI (Qt parent)]
│   └── CameraCardWidget × 4 (self.cards[i]) [Owner: ForkliftSafetyGUI (Qt parent)]
│       ├── AlarmSystem (self.alarm_system) [Owner: CameraCardWidget]
│       ├── VideoLabel (self.video_label) [Owner: CameraCardWidget (Qt parent)]
│       ├── CircularLED (self.led) [Owner: CameraCardWidget (Qt parent)]
│       ├── QTimer (self.blink_timer) [Owner: CameraCardWidget (Qt parent)]
│       └── CameraWorkerThread (optional) [Owner: CameraCardWidget]
│           ├── Camera (self.camera) [Owner: CameraWorkerThread]
│           └── cv2.VideoCapture (self.cap) [Owner: Camera]
```

22.2 Object Ownership Tree — Process 2 (hardware_worker.py)

```
main() [Python runtime]
├── SharedState (state) [Owner: main()]
│   ├── threading.Lock (self._lock) [Owner: SharedState]
│   └── dict (self._data) [Owner: SharedState]
├── IPCServer (ipc_server) [Owner: main()]
│   ├── socket.socket (self.server_socket) [Owner: IPCServer]
│   ├── list (self.clients) [Owner: IPCServer]
│   ├── threading.Lock (self.clients_lock) [Owner: IPCServer]
│   └── Thread (per client, daemon) [Owner: OS – daemon threads]
├── Thread (worker_thread, daemon) [Owner: main()]
│   └── [HardwareWorker – stack local in run_worker_thread()]
│       ├── serial.Serial (ser) [Owner: HardwareWorker]
│       └── dict (last_sent_state) [Owner: HardwareWorker]
```

22.3 Object Lifetime Table

Object	Created	Destroyed	Explicit Cleanup
<code>QApplication</code>	<code>main()</code> in <code>ai_gui_system.py</code>	<code>app.exec_()</code> returns	None needed (Qt handles it)
<code>ForkliftSafetyGUI</code>	<code>main()</code>	When <code>QApplication</code> event loop ends	<code>closeEvent()</code> stops cameras, disconnects relay
<code>YoloDetector</code>	<code>ForkliftSafetyGUI.__init__()</code>	When <code>ForkliftSafetyGUI</code> garbage collected	None (model unloads from RAM)
<code>USBRelayManager</code>	<code>ForkliftSafetyGUI.__init__()</code>	<code>closeEvent()</code> → <code>relay_manager.cleanup()</code>	<code>cleanup()</code> closes socket, stops recv thread
<code>CameraWorkerThread</code>	<code>start_camera()</code>	<code>stop_camera()</code> or <code>closeEvent()</code>	<code>stop()</code> → sets <code>running=False</code> → <code>wait(3000)</code>
<code>Camera</code> (in thread)	<code>CameraWorkerThread.run()</code>	When <code>CameraWorkerThread.run()</code> returns	<code>camera.release()</code> → <code>cap.release()</code>
<code>cv2.VideoCapture</code>	<code>Camera.initialize_camera()</code>	<code>Camera.release()</code>	<code>cap.release()</code>
<code>SharedState</code>	<code>hardware_worker.main()</code>	Process exit	None (OS reclaims memory)
<code>serial.Serial</code>	<code>open_serial_connection()</code>	<code>HardwareWorker._teardown()</code> or <code>run_worker_thread()</code>	<code>ser.close()</code>
<code>IPCTServer.server_socket</code>	<code>IPCTServer.start()</code>	Never explicitly (process exit)	OS closes file descriptors on exit
YOLO PyTorch tensors	YOLO model load	When <code>YoloDetector</code> garbage collected	None (PyTorch reference counting)

22.4 Memory Leak Analysis

Identified Potential Leaks:

- Old `YoloDetector` after model path change:** When the user changes the YOLO model path in Config and saves, `save_global_configurations()` creates a new `YoloDetector` and assigns it to `self.detector` and all `card.detector`. The old `YoloDetector` object has its reference count drop to 0 — Python GC will free it. The

PyTorch tensors inside will also be freed. This is safe, **but** model unload is not instantaneous — peak RAM may briefly be 2× model size during the switch.

2. **CameraWorkerThread** after **stop_camera()** **timeout**: If `thread.wait(3000)` times out (thread has not exited within 3 seconds), the thread continues executing while the card may no longer be alive. This is mitigated by `self.running = False` — the thread will exit at the next loop iteration. The Qt parent ownership ensures the widget memory stays alive until Qt garbage collects it.
 3. **clients** list in **IPCServer** : Dead client sockets are only purged when `broadcast_status()` detects a send failure. If the GUI disconnects without sending a `disconnect` command, the dead socket file descriptor lingers in the `clients` list until the next status broadcast. Maximum lifetime: until the next `hw_status` change.
 4. **Audit log ring buffer**: `log_terminal` is capped at 50 items via `takeItem(0)` when count exceeds 50. This is correct and prevents unbounded growth. No leak.
 5. **recv_buffer** in **_receiver_loop** : If a very large malformed message is received (never terminated by `\n`), `recv_buffer` accumulates indefinitely. In practice, all messages are well-formed JSON + `\n`, so this is a theoretical risk only.
-

SECTION 23 — COMPLETE FUNCTION CALL GRAPH

23.1 Process Startup Call Graph

```

start_system.sh
├─ [bash] export DISPLAY, XAUTHORITY, xhost
├─ [bash] pkill hardware_worker.py
├─ [bash] rm -f /dev/shm/forklift_relay.sock
├─ [bash] source venv/bin/activate
├─ hardware_worker.py → main()
│   └─ check_singleton()
│       └─ fcntl.flock(lock_fd, LOCK_EX|LOCK_NB)
├─ load_initial_config()
│   └─ os.path.join(Script_DIR, "config.json")
│   └─ json.load(f)
├─ SharedState.__init__()
│   └─ threading.Lock()
│   └─ load_initial_config() [called again internally]
│   └─ dict() initialization with mapping
├─ IPCServer.__init__(state, SOCKET_PATH)
│   └─ socket.socket(AF_UNIX, SOCK_STREAM)
│   └─ socket.bind(SOCKET_PATH)
│   └─ os.chmod(SOCKET_PATH, 0o660)
│   └─ socket.listen(5)
├─ Thread(target=ipc_server.start, daemon=True).start()
│   └─ IPCServer.start()
│       └─ [loop] server_socket.accept()
│           └─ Thread(target=handle_client, daemon=True).start()
│               └─ IPCServer.handle_client(conn)
│                   └─ [loop] conn.recv(4096)
│                       └─ IPCServer.process_command(cmd, conn)
│                           └─ [trigger] SharedState.set(station_N_trigger, val)
│                           └─ [config] SharedState.update({...})
│                           └─ [relay_mapping] SharedState.update({station_N_relay:...})
│                           └─ [connect] SharedState.reset_triggers(); broadcast_status()
│                           └─ [disconnect] SharedState.reset_triggers()
│                           └─ [status] broadcast_status()
├─ Thread(target=run_worker_thread, args=(state, ipc_server), daemon=True).start()
│   └─ run_worker_thread(shared_state, ipc_server)
│       └─ [loop] get_serial_port()
│           └─ os.path.exists(UDEV_RELAY_PATH)
│           └─ get_config_port()
│               └─ json.load(config.json)
│               └─ glob.glob("/dev/ttyUSB*")
│       └─ open_serial_connection(port)
│           └─ is_port_present(port)
│           └─ serial.Serial(port, BAUD_RATE, timeout=1)
│           └─ time.sleep(2) # CH340 stabilization
│       └─ [if boot_self_test] run_boot_self_test(ser)
│           └─ [ch=1..4] ser.write(b"Nch"); sleep; ser.write(b"Fch"); sleep
│       └─ shared_state.set("hw_status", "ONLINE")
│       └─ ipc_server.broadcast_status()
│       └─ HardwareWorker.__init__(ser, state, ipc)
│           └─ HardwareWorker.run()
│               └─ [every 50ms] state.get_snapshot()
│               └─ Phase 1: build station_states dict
│               └─ Phase 2: OR reduce → targets dict
│               └─ Phase 3: compare vs last_sent_state
│                   └─ HardwareWorker.transmit(ch, on)
│                       └─ ser.write(b"Nch") or ser.write(b"Fch")
│                       └─ ser.flush()
│               └─ time.sleep(POLL_INTERVAL)
├─ [bash] poll /dev/shm/forklift_relay.sock (every 0.5s, max 15s)
├─ ai_gui_system.py → main()
│   └─ QApplication(sys.argv)
│   └─ ForkliftSafetyGUI.__init__()
│       └─ SecurityManager.__init__()
│           └─ create_file_if_not_exists()
│               └─ json.dump(default_passwords, file)
│       └─ load_global_settings()
│           └─ json.load("config.json")
│       └─ [for each /dev/video0,2,4,6] os.path.exists()

```

```

└─ YoloDetector.__init__(model_path, confidence)
    └─ os.path.exists(model_path)
        └─ YOLO(model_path) # ~1-3 second model load
└─ USBRelayManager.__init__()
    └─ threading.Lock() × 2
    └─ Thread(target=_connect_socket, daemon=True).start()
        └─ _connect_socket()
            └─ socket.socket(AF_UNIX, SOCK_STREAM)
                └─ socket.connect(SOCKET_PATH)
└─ [build entire PyQt5 widget tree]
└─ CameraCardWidget.__init__() × 4
    └─ AlarmSystem.__init__()
    └─ VideoLabel.__init__()
    └─ CircularLED.__init__()
    └─ load_roi()
        └─ json.load("config.json")
└─ relay_manager.connect_port(com_port)
    └─ _send_command({cmd:"connect",...})
└─ global_start_all()
    └─ [× 4] CameraCardWidget.start_camera()
        └─ CameraWorkerThread.__init__(device_index)
            └─ CameraWorkerThread.start()
                └─ CameraWorkerThread.run() [new thread]
└─ installEventFilter(self) # for inactivity detection
└─ inactivity_timer.start(300000)
└─ status_check_timer.start(1000)
    └─ poll_hardware_status() [every 1 second]
        └─ relay_manager.hw_status [property]
└─ app.exec_() [enters Qt event loop]

```

23.2 Per-Frame Processing Call Graph (critical path, ~30 FPS)

```

CameraWorkerThread.run()
└─ camera.read_frame() # V4L2 blocking call
    └─ cv2.VideoCapture.read() # ~33ms blocking
└─ emit frame_ready(frame) # Qt signal
    └─ [Qt marshals to main thread]
        └─ CameraCardWidget.on_frame_received(frame)
            └─ frame.shape # get dims
            └─ [compute scale_x, scale_y]
            └─ [scale ROI]
            └─ frame_counter += 1
            └─ [if frame_counter % 3 == 0]
                └─ YoloDetector.detect_persons(frame, imgsz=320)
                    └─ self.model(frame, conf=..., imgsz=320, verbose=False)
                        └─ [YOLO CSP inference - ~100-400ms on CPU]
                └─ AlarmSystem.check_boundaries(persons, scaled_roi)
                    └─ [for each person] QRect.contains(QPoint)
            └─ [check relay_manager.manual_overrides[cam_id]]
            └─ [if warning != last_hardware_state]
                └─ relay_manager.trigger_relay(cam_id, warning)
                    └─ [latch check]
                    └─ [cooldown check]
                    └─ _send_command({cmd:"trigger",...})
                        └─ socket.sendall(json + "\n")
            └─ [draw bboxes on frame] cv2.rectangle(), cv2.putText()
            └─ [draw ROI overlay] cv2.addWeighted()
            └─ [calculate FPS] time.time()
            └─ cv2.cvtColor(frame, BGR2RGB)
            └─ QImage(frame_rgb.data, w, h, ...)
            └─ video_label.setPixmap(QPixmap.fromImage(qimage))
            └─ [update LED, banner, metrics labels]
            └─ parent_window.evaluate_global_alarms()
                └─ [update footer status bar]

```

23.3 Shutdown Call Graph

```

User triggers close (Alt+F4 or X button or Exit button)
├─ ForkliftSafetyGUI.closeEvent(event)
│   ├─ [check role_level < 3] → event.ignore() [ABORT]
│   └─ [role_level >= 3]
│       ├─ add_audit_log("System closing...")
│       ├─ [for each card] CameraCardWidget.stop_camera()
│       │   ├─ blink_timer.stop()
│       │   └─ CameraWorkerThread.stop()
│       │       ├─ self.running = False
│       │       └─ self.wait(3000) # wait for thread exit
│       │           └─ CameraWorkerThread.run() exits loop
│       │               └─ Camera.release()
│       │                   └─ cv2.VideoCapture.release()
│       └─ camera_thread.deleteLater() [Qt deferred delete]
│           └─ relay_manager.disconnect()
│               └─ _send_command({cmd:"disconnect"})
│                   └─ SharedState.reset_triggers() [server side]
│   └─ relay_manager.cleanup()
│       ├─ _recv_running = False
│       ├─ _socket.shutdown(SHUT_RDWR)
│       └─ _socket.close()
│   └─ event.accept()
│       └─ QApplication.exit(0)
│           └─ Python runtime cleanup
│               └─ all objects garbage collected

[start_system.sh resumes]
├─ kill $HW_PID
│   └─ hardware_worker.py receives SIGTERM
│       └─ [daemon threads receive KeyboardInterrupt]
│           └─ HardwareWorker.run() KeyboardInterrupt handler
│               ├─ [for each last_sent_state[ch] == 1]
│               │   └─ transmit(ch, False) # turn OFF active relays
│               └─ return True # clean exit signal
└─ rm -f /dev/shm/forklift_relay.sock

```


SECTION 24 — VARIABLE FLOW ANALYSIS

24.1 Camera Frame Variable Flow

CREATION

```
V4L2 kernel driver allocates frame buffer in kernel memory
↓
cv2.VideoCapture.read() copies kernel buffer → numpy.ndarray
  Type: numpy.ndarray, dtype=uint8, shape=(480, 640, 3), colorspace=BGR
  Memory: ~921,600 bytes per frame
  Owner: CameraWorkerThread (local variable in run())
↓
frame is passed to Qt signal emit(frame)
Qt copies the frame reference into the signal delivery queue
(NOT a deep copy – same numpy buffer)
```

DELIVERY

```
Qt delivers signal to main thread slot
→ on_frame_received(frame) [main thread]
The numpy.ndarray is now accessible in the main thread
(Safe due to Qt's signal queuing – delivery is serialized)
```

MODIFICATION

1. cv2.rectangle() called on frame [in-place mutation]
Draws bounding boxes directly onto the numpy array
2. cv2.addWeighted(overlay, alpha, frame, ...) [in-place blend]
Blends ROI highlight onto frame
3. cv2.putText() called [in-place mutation]
Draws FPS text, station labels
4. cv2.cvtColor(frame, BGR2RGB) [creates new array]
Returns new numpy array – frame_rgb
Original frame array unchanged

CONVERSION

```
QImage(frame_rgb.data, w, h, ch*w, QImage.Format_RGB888)
  Creates QImage backed by frame_rgb memory (shallow)
  .copy() may be called → deep copy to independent buffer
```

DISPLAY

```
QLabel.setPixmap(QPixmap.fromImage(qimage))
  Qt renders pixmap to screen
```

DESTRUCTION

```
After on_frame_received() returns:
- frame, frame_rgb, overlay: reference count drops → GC
- QImage: may be held by Qt rendering pipeline briefly
- QPixmap: held by QLabel until next frame replaces it
Total frame lifetime: ~33ms between frames
```

24.2 Warning State Variable Flow

CREATION

```
AlarmSystem.check_boundaries(persons, roi_rect)
→ returns bool (True if person center inside ROI)
Variable: warning: bool
Type: bool
Owner: CameraCardWidget.on_frame_received() [local]
```

COMPARISON (edge detection)

```
if warning != self.last_hardware_state:
→ Only proceeds if state has changed
→ Prevents redundant IPC messages (critical optimization)
```

TRANSMISSION

```
USBRelayManager.trigger_relay(cam_id, warning)
→ warning (bool) converted to 1/0 in JSON cmd
→ JSON: {"cmd":"trigger","station":N,"state":1/0}
→ socket.sendall(json_bytes)
```

SERVER SIDE

```
IPCServer._process_command() receives cmd
→ SharedState.set("station_N_trigger", 1/0)
→ Value stored in SharedState._data (int)
```

CONSUMPTION

```
HardwareWorker.run() reads snapshot every 50ms
→ station_states[N] = 1/0
→ OR reduction → targets[ch] = 1/0
→ if targets[ch] != last_sent_state[ch]:
    transmit(ch, True/False)
→ ser.write(b"Nch" or b"Fch")
```

SIDE EFFECTS (GUI)

```
warning = True:
→ led.set_state("breach") [flash timer starts]
→ card frame border → red
→ card property("warning") = "true"
→ parent_window.evaluate_global_alarms() → footer red
```

```
warning = False:
→ led.set_state("secure") [flash timer stops]
→ card frame border → green
→ footer returns to green
```

STORAGE

```
self.last_hardware_state = warning [after successful send]
self.last_warning = warning [for frame skip cache]
self.prev_warning = warning [for transition logging]
```

24.3 ROI Coordinate Variable Flow

CREATION (First time)

User draws ROI on VideoLabel with mouse
 → VideoLabel.mousePressEvent() → is_drawing = True
 → VideoLabel.mouseMoveEvent() → self.roi_rect updated
 → VideoLabel.mouseReleaseEvent() → emit roi_updated(QRect)
 Type: QRect(x, y, w, h) in widget pixel space

PERSISTENCE

CameraCardWidget.save_roi(roi_rect)
 → json.load("config.json")
 → data["camera_N"] = {"x":..., "y":..., "w":..., "h":...}
 → json.dump(data, "config.json")
 Values are widget-space pixels (NOT frame pixels)

LOADING

CameraCardWidget.load_roi()
 → json.load("config.json")["camera_N"]
 → QRect(x, y, w, h) [widget pixel space]
 → self.video_label.set_roi(roi_rect)

SCALING (per frame)

```
on_frame_received(frame):
    widget_w = self.video_label.width()
    widget_h = self.video_label.height()
    scale_x = frame.shape[1] / widget_w    # typically 640 / ~400
    scale_y = frame.shape[0] / widget_h    # typically 480 / ~300
    scaled_roi = QRect(
        roi.x() * scale_x,
        roi.y() * scale_y,
        roi.width() * scale_x,
        roi.height() * scale_y
    )
    → scaled_roi is in FRAME pixel space (0-640, 0-480)
```

USAGE

AlarmSystem.check_boundaries(persons, scaled_roi)
 → QRect.contains(QPoint(center_x, center_y))
 → All coordinates are in frame pixel space – consistent

RESOLUTION: In the recent production upgrade, this limitation was completely resolved by implementing Normalized Relative Coordinates (0.0 to 1.0):
 self.norm_roi = (norm_x, norm_y, norm_w, norm_h).
 A custom resizeEvent in VideoLabel dynamically recalculates and re-scales the pixel bounding box to match the current widget dimensions seamlessly, guaranteeing zero visual drift on window resize or resolution changes.
 (See Section 31.6 for detailed architecture).

24.4 IPC Message Variable Flow

CREATION

```
USBRelayManager.trigger_relay(camera_id=0, state=True)
    channel_index = 1
    cmd = {"cmd": "trigger", "station": 1, "state": 1}
    json_str = json.dumps(cmd) + "\n" → '{"cmd": "trigger", "station": 1, "state": 1}\n'
    json_bytes = json_str.encode("utf-8") → b'{"cmd": "trigger", "station": 1, "state": 1}\n'
```

TRANSMISSION

```
socket.sendall(json_bytes)
    → OS copies bytes to socket send buffer (kernel)
    → Kernel routes through Unix Domain Socket (in-memory, zero-copy)
    → Server recv buffer receives bytes
```

RECEPTION (hardware_worker.py IPCServer._handle_client)

```
conn.recv(4096) → raw bytes
recv_buffer += data.decode("utf-8")
[find \n] → line = '{"cmd": "trigger", "station": 1, "state": 1}'
cmd = json.loads(line)
    → Python dict: {"cmd": "trigger", "station": 1, "state": 1}
```

PROCESSING

```
cmd["cmd"] == "trigger"
station = int(cmd["station"]) → 1
state = int(cmd.get("state", 0)) → 1
shared_state.set("station_1_trigger", 1)
    → acquires _lock → _data["station_1_trigger"] = 1 → releases _lock
```

CONSUMPTION (next OR-gate cycle, ≤50ms later)

```
HardwareWorker.run() → state.get_snapshot()
    → {"station_1_trigger": 1, ...}
station_states[1] = 1
targets[relay_ch] = 1 [if station_1_relay == relay_ch]
if targets[ch] != last_sent_state[ch]:
    transmit(ch, True)
    → ser.write(b"N3") [if relay_ch == 3]
    → ser.flush()
    → Physical relay closes contact
```

DESTRUCTION

```
json_bytes: freed after sendall() (reference drops)
cmd dict: freed after processing
state persists in SharedState._data until next trigger sets it to 0
```

SECTION 31 — RECENT PRODUCTION-GRADE UPGRADES & SYSTEM RESILIENCE

To achieve 99.99% industrial uptime and satisfy Pharma 4.0 operational standards in harsh warehouse environments, the codebase underwent a comprehensive production resilience upgrade. These upgrades systematically eliminate physical vibration failures, hardware enumeration race conditions, layout distortion, and unexpected crashes.

31.1 Physical USB Port Binding & Anti-Swapping Architecture

Problem Statement

Industrial forklifts operate in high-vibration environments. Vibration frequently causes micro-disconnects on USB camera cables. When the Linux kernel re-detects a camera after a momentary disconnect, V4L2 does not guarantee re-assignment to the same `/dev/videoX` node. For example, a rear-view camera previously assigned to `/dev/video0` may be re-enumerated as `/dev/video8`. Under the legacy system, this caused catastrophic camera station swapping, displaying the rear view on the front camera card and corrupting safety boundaries.

Technical Solution

- Kernel Topology Path Resolution (`get_physical_usb_path`):** Implemented `get_physical_usb_path(dev_path)` in `ai_gui_system.py`. Instead of relying on mutable `/dev/videoX` filenames, the system resolves the physical sysfs device symlink: `python # Reads kernel hardware topology: # /sys/class/video4linux/videoX/device -> ../../../../1-1.2:1.0 real_device = os.path.realpath(f"/sys/class/video4linux/{dev_name}/device")`. This extracts the invariant USB physical port address on the motherboard or industrial USB hub (e.g., `1-1.2` for physical port 1, subport 2).
- Global Thread-Safe Device Registry (`_registry_lock`):** A module-level re-entrant lock protects camera acquisition across all `CameraWorkerThread` instances. Before a worker attempts to bind a camera node, it verifies both the physical USB topology path and ensures no other station has an active handle on that hardware unit.
- Automated Recovery:** If a camera disconnects and reconnects on a different `/dev/videoX` index, the background worker scans available devices, matches the physical USB hub port signature, and automatically binds the correct stream back to its designated station without operator intervention.

31.2 Station-Level Relay Mapping & Interactive Live Testing

Problem Statement

Operators and maintenance engineers needed immediate visual clarity and runtime control over which physical relay channel (Relay 1 through Relay 4) each camera station controls, as well as an instant method to test sirens or strobe lights directly from the live monitoring view without navigating deep into administrative menus.

Technical Solution

- Integrated Relay Channel Selector (⚡ `Relay` : Dropdown):** Each camera card (`CameraCardWidget`) is equipped with a dedicated dropdown selector on its control bar: - Populated with options: `Relay 1`, `Relay 2`, `Relay 3`, `Relay 4`. - Connected to `on_relay_changed()` signal. - Synchronizes bidirectionally with `config.json` (`relay_mapping.camera_X`) and dispatches an instant IPC update to the background `hardware_worker.py` daemon.
- Interactive Live Test Button (⚡ `TEST R1 - R4`):** Added an instant toggle test button to every camera card: - Dynamically updates its label to match the currently assigned relay channel (e.g., ⚡ `TEST R2`). - Toggles hardware test mode directly through `USBRelayManager.set_manual_test()`. - Provides immediate visual feedback: turns glowing crimson red (`#dc2626`) during active test, signaling that the physical relay contact is forced closed. - Self-contained and operable directly from the Kiosk view by authorized personnel.

31.3 Systemd Auto-Restart & Self-Healing Service Architecture

Problem Statement

Industrial safety systems must recover autonomously from any software fault, unhandled exception, or unexpected X11 crash. Previously, the system was launched via desktop shortcuts or generic shell scripts, creating race conditions where multiple instances could launch simultaneously or fail to restart after an unexpected termination.

Technical Solution

1. **Dedicated User-Session Systemd Service (`forklift-ai.service`)**: Configured at `~/.config/systemd/user/forklift-ai.service`: `ini [Unit] Description=AI Forklift Safety System (GUI & Hardware Worker) After=graphical-session.target sound.target Wants=graphical-session.target`

```
[Service] Type=simple WorkingDirectory=/home/forklift/Desktop/AI_Forklift_Safety ExecStart=/bin/bash /home/forklift/Desktop/AI_Forklift_Safety/start_system.sh --no-delay Restart=always RestartSec=3 TimeoutStartSec=60 KillMode=mixed Environment=DISPLAY=:0 Environment=XAUTHORITY=/home/forklift/.Xauthority Environment=QT_QPA_PLATFORM=xcb
```

```
[Install] WantedBy=graphical-session.target `` 2. **Deterministic Auto-Recovery ( Restart=always , RestartSec=3 ):** If the GUI process, Python runtime, or hardware daemon terminates for any reason, Systemd automatically restarts the entire safety stack within 3 seconds. 3. **User-Session Lingering ( loginctl enable-linger forklift ):** Enabled lingering for the forklift user account, ensuring systemd user services initialize at machine boot before user login and continue running uninterrupted. 4. **Unified Launch Path:** All desktop shortcuts and autostart launchers ( ~/.config/autostart/AI_Forklift_Safety.desktop and /home/forklift/Desktop/run_system.sh ) were unified to issue systemctl --user restart forklift-ai.service`. This strictly prevents dual-instance collisions and race conditions.
```

31.4 15-Second Hardware & USB Bus Stabilization Delay

Problem Statement

At cold machine boot, modern motherboards and industrial USB hubs experience power surges and staggered enumeration. Linux kernel modules (`uvcvideo` and `ch341`) take several seconds to register all video nodes and serial devices. Launching the application immediately on boot caused intermittent failures where cameras 3 and 4 were missed or the relay board was not yet accessible at `/dev/forklift_relay`.

Technical Solution

1. **Live Countdown Sequence in `start_system.sh`**: At system boot, `start_system.sh` executes a 15-second hardware stabilization loop with live visual feedback: `bash echo "⌚ Waiting for Linux kernel & USB devices to stabilize..." for i in {15..1}; do echo -ne " Stabilizing hardware: $i seconds remaining...\r" sleep 1 done` This guarantees that all 4 camera V4L2 streams and the CH340 serial relay chip have settled electrically and registered in `/sys` before software binding begins.
2. **Developer & Automated Bypass Flag**: Supports immediate launch via `--no-delay` argument or `SKIP_DELAY=1` environment variable. When systemd restarts the service or an engineer runs tests, the delay is skipped cleanly: `bash ./start_system.sh --no-delay`

31.5 Layout Stretch & Aspect Ratio Protection

Problem Statement

When a camera cable was unplugged or signal lost, the camera card would replace the live video feed with an offline warning message. In the legacy UI, generating this offline banner with dynamic widget pixel dimensions triggered Qt's

layout engine to recalculate column and row sizes. As a result, the remaining active camera cards would suddenly stretch, expand, or warp their aspect ratios.

Technical Solution

1. **Locked 50/50 2x2 Grid Constraints:** Configured invariant stretch factors on the main grid layout: `python`
`self.grid_layout.setRowStretch(0, 1) self.grid_layout.setRowStretch(1, 1)`
`self.grid_layout.setColumnStretch(0, 1) self.grid_layout.setColumnStretch(1, 1)`
2. **Decoupled Size Policy (`QSizePolicy.Ignored`):** Set `self.video_label.setSizePolicy(QSizePolicy.Ignored, QSizePolicy.Ignored)`. This instructs Qt that the `VideoLabel` must strictly adapt to the layout's assigned space rather than requesting space based on incoming image or banner dimensions.
3. **Fixed Size Hints in `VideoLabel` (`roi.py`):** Overrode `sizeHint()` and `minimumSizeHint()` to return an invariant `QSize(320, 240)` (standard 4:3 aspect ratio).
4. **Standardized Offline Canvas (`640x480`):** `update_offline_banner()` now renders onto a fixed standard `640x480` canvas with centered warning text and icons, completely eliminating dimension shifts.

31.6 Normalized ROI Coordinates System (0.0 to 1.0) & Dynamic Rescaling

Problem Statement

In the original architecture, Region of Interest (ROI) boundaries were stored in `config.json` as absolute pixel coordinates relative to the widget size at the time of drawing (e.g., `x: 142, y: 88, w: 220, h: 180`). If the application window was resized, or if the system ran on a display with a different resolution, the bounding box would visually drift or misalign relative to the underlying video stream. Furthermore, corrupted configurations (such as width exceeding frame dimensions) could trigger crashes.

Technical Solution

1. **Precision Normalized Ratio System (`norm_roi`):** `VideoLabel` in `roi.py` now converts all ROI coordinates into normalized floating-point ratios between `0.0` and `1.0`: $\text{norm_x} = \frac{x}{\text{widget_width}}$, $\text{norm_y} = \frac{y}{\text{widget_height}}$, $\text{norm_w} = \frac{w}{\text{widget_width}}$, $\text{norm_h} = \frac{h}{\text{widget_height}}$
2. **Dynamic `resizeEvent()` Recalculation:** Whenever the window or card is resized, `VideoLabel.resizeEvent()` automatically recomputes the pixel bounding box: `python`
`def resizeEvent(self, event):`
`super().resizeEvent(event) if self.norm_roi: nx, ny, nw, nh = self.norm_roi w = self.width() h =`
`self.height() self.roi_rect = QRect(int(nx * w), int(ny * h), int(nw * w), int(nh * h))`
`self.update()`
3. **Direct YOLO Frame Mapping:** When evaluating safety violations in `CameraCardWidget.on_frame_received()`, the normalized coordinates map directly to incoming OpenCV frame dimensions (`frame_w = frame.shape[1]`, `frame_h = frame.shape[0]`): $\text{scaled_roi} = \text{QRect}(\text{norm_x} \cdot W_{\text{frame}}, \text{norm_y} \cdot H_{\text{frame}}, \text{norm_w} \cdot W_{\text{frame}}, \text{norm_h} \cdot H_{\text{frame}})$. This ensures absolute mathematical precision regardless of display scaling, widget resizing, or camera resolution.
4. **Configuration Sanitization:** Corrected out-of-bounds legacy values in `config.json` (such as `w: 723` on a 640-pixel stream), guaranteeing safe boundary clamps `[0.0, 1.0]`.

31.7 Universal Responsive UI Scaling (800x480 to 4K UHD)

Problem Statement

Forklifts use diverse in-cab display hardware, ranging from small 7-inch embedded touchscreens (\$800 \times 480\$ or \$1024 \times 600\$) to full HD (\$1920 \times 1080\$) and ultra-high-definition 4K monitors (\$3840 \times 2160\$). Fixed font sizes, hardcoded margins, and rigid minimum window dimensions previously caused UI clipping on small screens and microscopic elements on 4K displays.

Technical Solution

- 1. **Adaptive Dynamic Scale Factor (`_ui_scale`)**: Calculated dynamically based on active screen resolution:
$$\text{scale} = \min\left(\frac{W_{\text{screen}}}{1920}, \frac{H_{\text{screen}}}{1080}\right), \quad \text{clamped to } [0.45, 1.50]$$
- 2. **Flexible Minimum Viewport Bounds**: - Base minimum window size adapts to: `min(_screen_w, 640), min(_screen_h, 400)` . - Card minimum dimensions scale smoothly:
$$\text{min_w} = \max(100, \lfloor 240 \cdot \text{scale} \rfloor), \quad \text{min_h} = \max(75, \lfloor 180 \cdot \text{scale} \rfloor)$$
- 3. **Multi-Resolution Validation**: The responsive architecture was simulated and verified across 9 distinct industry screen profiles:

Resolution	Aspect Ratio	Target Device Type	Result
800 × 480	5:3	Small embedded forklift in-cab display	✓ PASS (No clipping, legible layout)
1024 × 600	16:9	7" Industrial touchscreen panel	✓ PASS (Full 2x2 grid fits cleanly)
1024 × 768	4:3	Legacy industrial panel PC	✓ PASS (Optimal 4:3 aspect match)
1280 × 720	16:9	720p HD rugged vehicle terminal	✓ PASS (Balanced proportion)
1280 × 800	16:10	10.1" Rugged industrial tablet	✓ PASS (Excellent readability)
1366 × 768	16:9	Standard commercial laptop / monitor	✓ PASS (Standard desktop fit)
1920 × 1080	16:9	Full HD operator station	✓ PASS (Baseline reference standard)
2560 × 1440	16:9	2K QHD supervisory workstation	✓ PASS (Crisp fonts, expanded canvas)
3840 × 2160	16:9	4K UHD central control room wall	✓ PASS (High-DPI scaled, zero blurring)

31.8 Automated Validation Suites & Test Coverage

Five standalone automated validation test suites were developed in `scratch/` to verify every aspect of the production architecture. All suites run headless and achieve 100% pass rates:

- 1. `test_roi_and_layout_stability.py` (5/5 PASS): - Verifies invariant `sizeHint()` (\$320 \times 240\$). - Verifies `QSizePolicy.Ignored` decoupling on `VideoLabel` . - Validates normalized coordinate storage ($0.0 \leq \text{coord} \leq 1.0$). - Verifies dynamic `resizeEvent` recalculation without coordinate drift. - Tests offline banner dimensions (\$640 \times 480\$) and grid stretch stability.
- 2. `test_multi_resolution_adaptation.py` (9/9 PASS): - Simulates layout instantiation across all 9 target resolutions. - Confirms zero layout overflow, no overlapping widgets, and positive viewport areas.
- 3. `test_card_relay_and_camera_dropdown.py` (4/4 PASS): - Verifies relay channel dropdown initialization and bidirectional config synchronization. - Verifies interactive `⚡ TEST R1-R4` button state toggling and visual color transitions. - Validates camera device dropdown enumeration and physical USB port binding.

-
4. `test_production_architecture.py` (7/7 PASS): - Tests IPC socket server-client communication over `/dev/shm`. - Validates stateless OR-gate 50ms evaluation logic. - Confirms zero redundant serial transmissions on steady state.
 5. `test_physical_usb_binding.py` (7/7 PASS): - Tests sysfs USB hub topology resolution. - Validates global thread-safe registry locking (`_registry_lock`). - Simulates disconnect and reconnect on alternate `/dev/videoX` nodes.
-

31.9 Industrial Defect Hardening & Production Verification Summary

During the final production review (September 2026), 8 identified software defects were systematically resolved and validated to ensure 100% industrial reliability:

1. **YOLO Timer Relocation:** Moved inference and blink timer initialization from `on_physical_path_discovered` into `start_camera()` in `ai_gui_system.py`, guaranteeing real-time AI perception activates reliably upon system boot.
 2. **Defensive Detector Constructor:** Enhanced `detector.py` to accept `model_name`, `model_path`, and `**kwargs`, preventing runtime `TypeError` crashes during configuration reload.
 3. **Array Initialization Defense:** Permanently initialized `self.manual_overrides = [False, False, False, False]` in `USBRelayManager.__init__()` in `hardware_manager.py`, eliminating initialization race conditions.
 4. **Clean Journal & CPU Protection:** Suppressed noisy OpenCV C++ V4L2 device open warnings via `cv2.setLogLevel(0)` and `OPENCV_LOG_LEVEL="SILENT"`, reducing background CPU overhead and preventing log file exhaustion.
 5. **Accurate Diagnostic Banners:** Disentangled camera feed offline indicators from path intrusion alarms in the HMI status bar, ensuring operators receive precise, unambiguous telemetry.
 6. **Dynamic Audit Logs:** Synchronized the RM04U off-delay audit messages with the runtime `alarm_auto_reset` setting.
 7. **Clean Dependencies:** Rebuilt `requirements.txt` with only the essential production dependencies, purged system-level apt packages, and established `.gitignore` rules for seamless GitHub collaboration.
-

Documentation updated: September 2026 | Limitless Future AI Forklift Safety Engineering Team.